

Networks

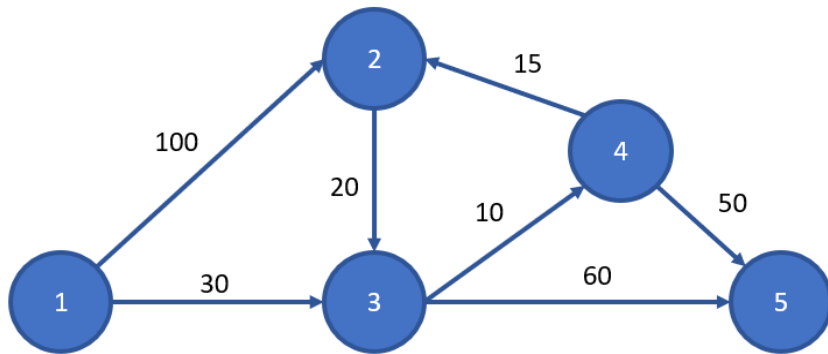
Shortest Paths and Minimum Spanning Tree (MST)

Network

Graph Topological

$E \subset V \times V$ $\downarrow$ $\{(1,2)\}$

Directed graph $G = (V, E)$, where E is a set of edges, subset of $V \times V$, and a weight function $w: E \rightarrow \mathbb{R}$



(i,j) $\textcircled{i} \longrightarrow \textcircled{j}$

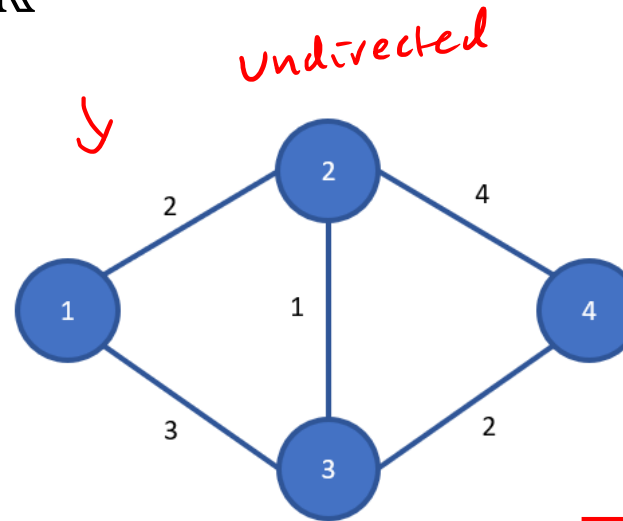
$V = \{1,2,3,4,5\}$

$E = \{(1,2), (1,3), (2,3), (3,4), (3,5), (4,2), (4,5)\}$

e.g. $w(1,2) = 100$

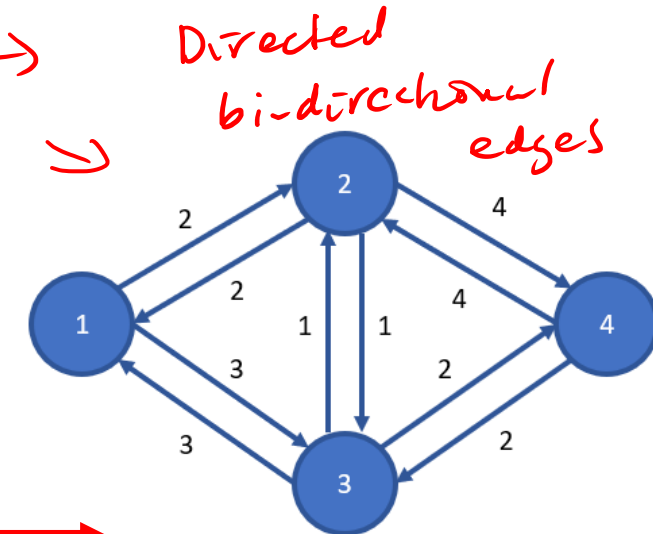
$w(i,j) = w_{ij}$

$w(3,4) = 10$



$V = \{1,2,3,4\}$

$E = \{(1,2), (1,3), (2,3), (2,4), (3,4)\}$



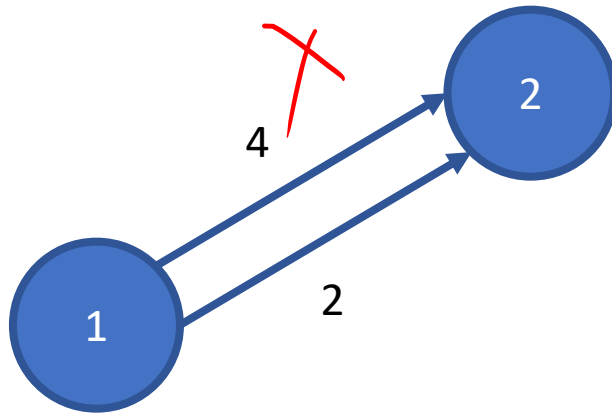
$V = \{1,2,3,4\}$

$E = \{(1,2), (1,3), (2,1), (2,3), (2,4), (3,1), (3,2), (3,4), (4,2), (4,3)\}$

✓

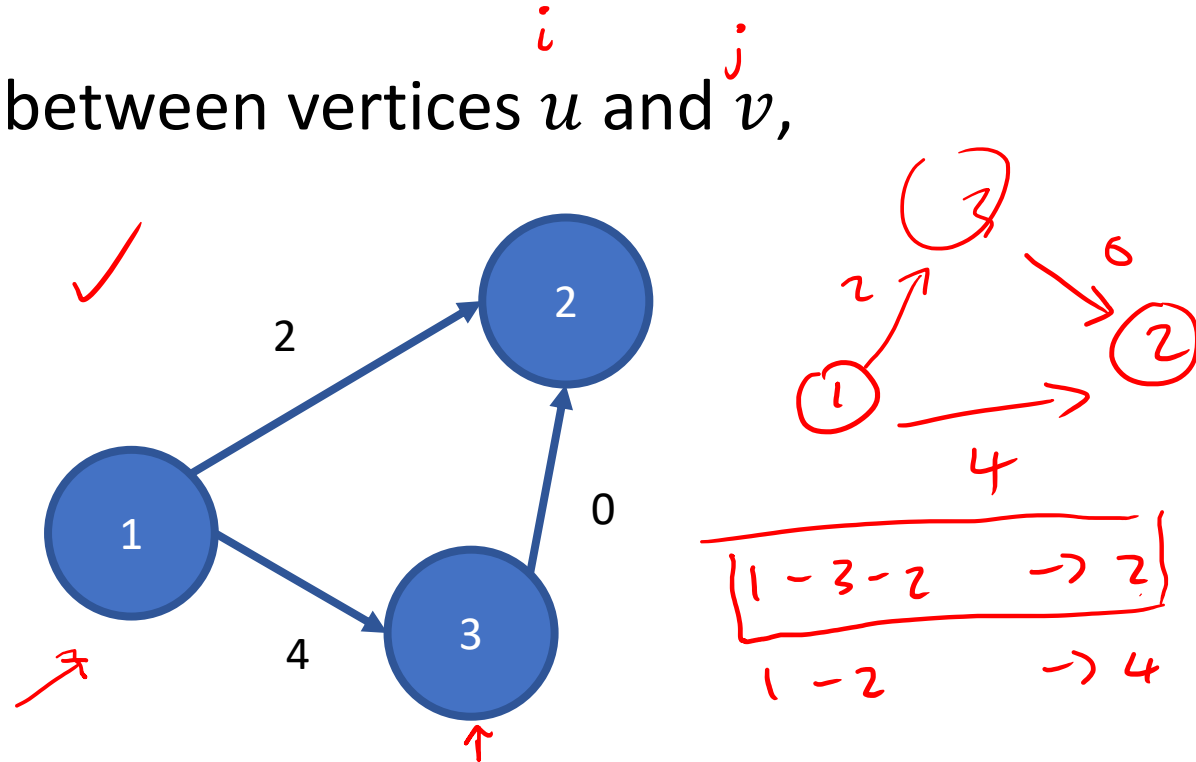
Network

Note we do not allow multiple edges between vertices u and v , otherwise it is not unique



What is the weight of $w(1,2)$?

Breaks the def. of a function



What is the weight of $w(1,2)$?

$$w(1,2) = 2$$

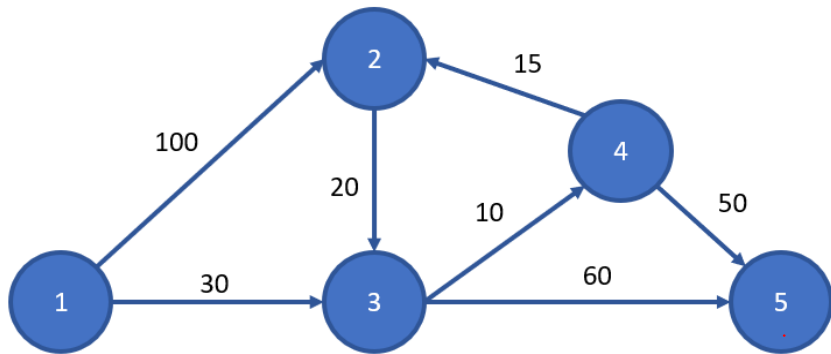
Alternative path from 1-2 is 1-3-2

$$w(1,3) + w(3,2) = 4 + 0 = 4$$

Adjacency Matrix

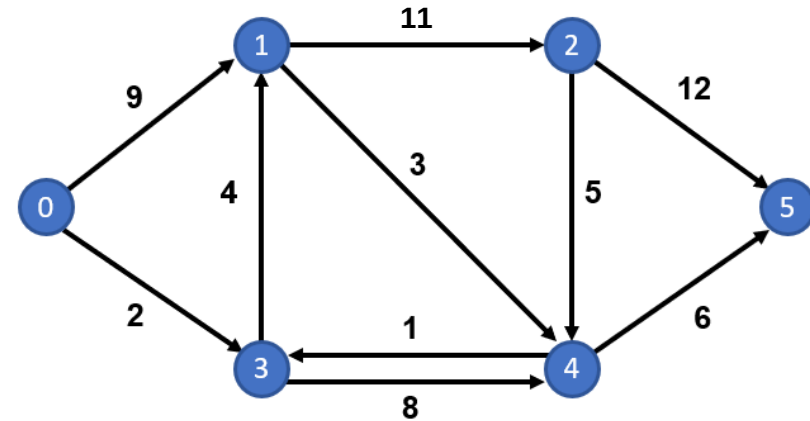
List

We won't have direct use for this, but it is worth noting!



1 → 2, 3
 2 → 1
 3 → 2, 1
 4 → 3
 5 → 4, 3

Paths



$$\begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{pmatrix} 0 & 100 & 30 & - & - \\ - & 0 & 20 & - & - \\ - & - & 0 & 10 & 60 \\ - & 15 & - & 0 & 50 \\ - & - & - & - & 0 \end{pmatrix}
 \end{matrix}$$

OR

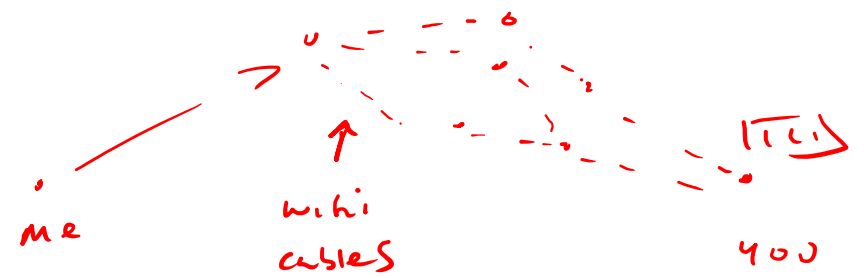
$$\begin{pmatrix} 0 & 100 & 30 & \infty & \infty \\ \infty & 0 & 20 & \infty & \infty \\ \infty & \infty & 0 & 10 & 60 \\ \infty & 15 & \infty & 0 & 50 \\ \infty & \infty & \infty & \infty & 0 \end{pmatrix}$$

$$\begin{pmatrix} 0 & 9 & - & 2 & - & - \\ - & 0 & 11 & - & 3 & - \\ - & - & 0 & - & 5 & 12 \\ - & 4 & - & 0 & 8 & - \\ - & - & - & 1 & 0 & 6 \\ - & - & - & - & - & 0 \end{pmatrix}$$

OR

$$\begin{pmatrix} 0 & 9 & \infty & 2 & \infty & \infty \\ \infty & 0 & 11 & \infty & 3 & \infty \\ \infty & \infty & 0 & \infty & 5 & 12 \\ \infty & 4 & \infty & 0 & 8 & \infty \\ \infty & \infty & \infty & 1 & 0 & 6 \\ \infty & \infty & \infty & \infty & \infty & 0 \end{pmatrix}$$

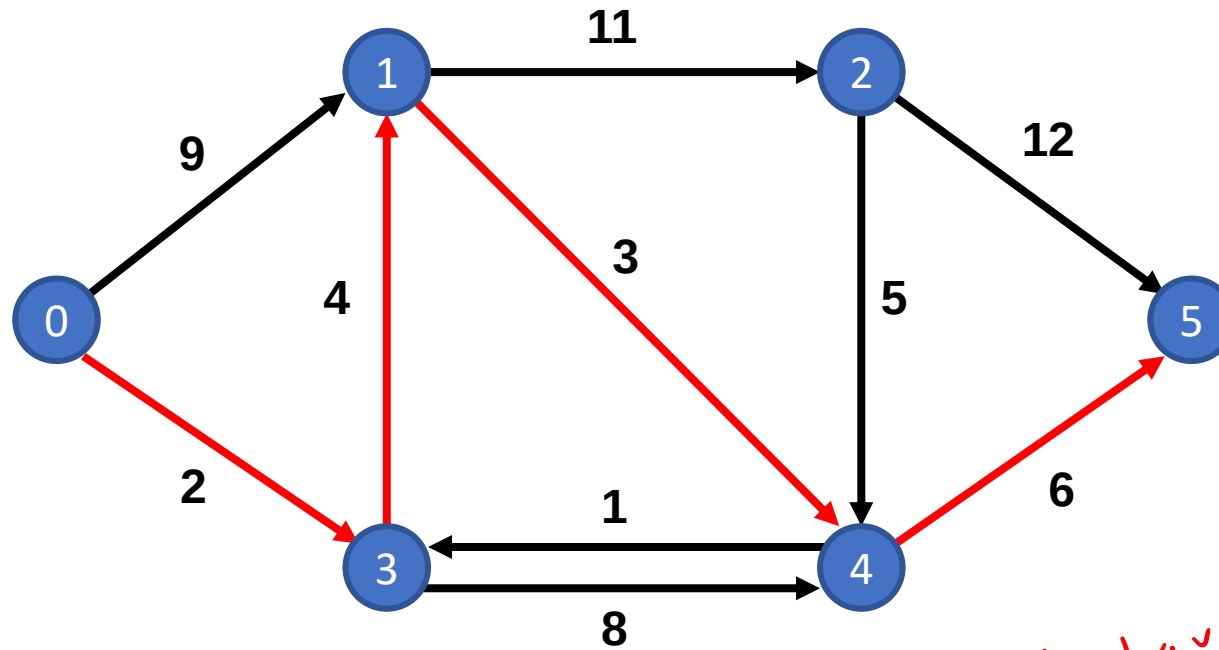
Shortest Path Problem



Definition: Find the shortest path between two vertices (source and destination) such that the sum of the weights of its edges are minimised.

- Perhaps one of the most fundamental problems in Mathematics and Computer Science
- Huge implications on routing and traffic problems, e.g. vehicular, telecommunications.
- If you've used Google Maps, you've asked to solve a form of shortest path problem!

Blue Peter Example



Longest Path

CRITICAL PATHS
DAG

SP ✓
LP

SP it we don't have negative weight cycles

Shortest path between 0 and 5 is 0-3-1-4-5 of length 15

Dijkstra's Algorithm

- Originally designed to test the limitations of computing power by Edsger Wybe Dijkstra in 1956
- Still widely used and researched today!
- Google uses an advanced pre-processing speed-up technique known as contraction hierarchies.



Edges weights need to be nonnegative!

Homework: Investigate Bellman-Ford Algorithm for detecting negative cycles

DIAG

1. Initialise the network, source vertex has a distance of 0 and all other vertices a distance of ∞ , label all vertices as **temporary**
2. Label the vertex with minimum distance as **permanent**
3. Update the distances and predecessors to reachable **temporary** vertices from the current permanent vertex
4. Repeat steps 2 and 3 until all vertices are marked **permanent**

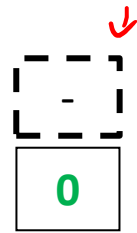
Set Up

Shortest Path
s - t

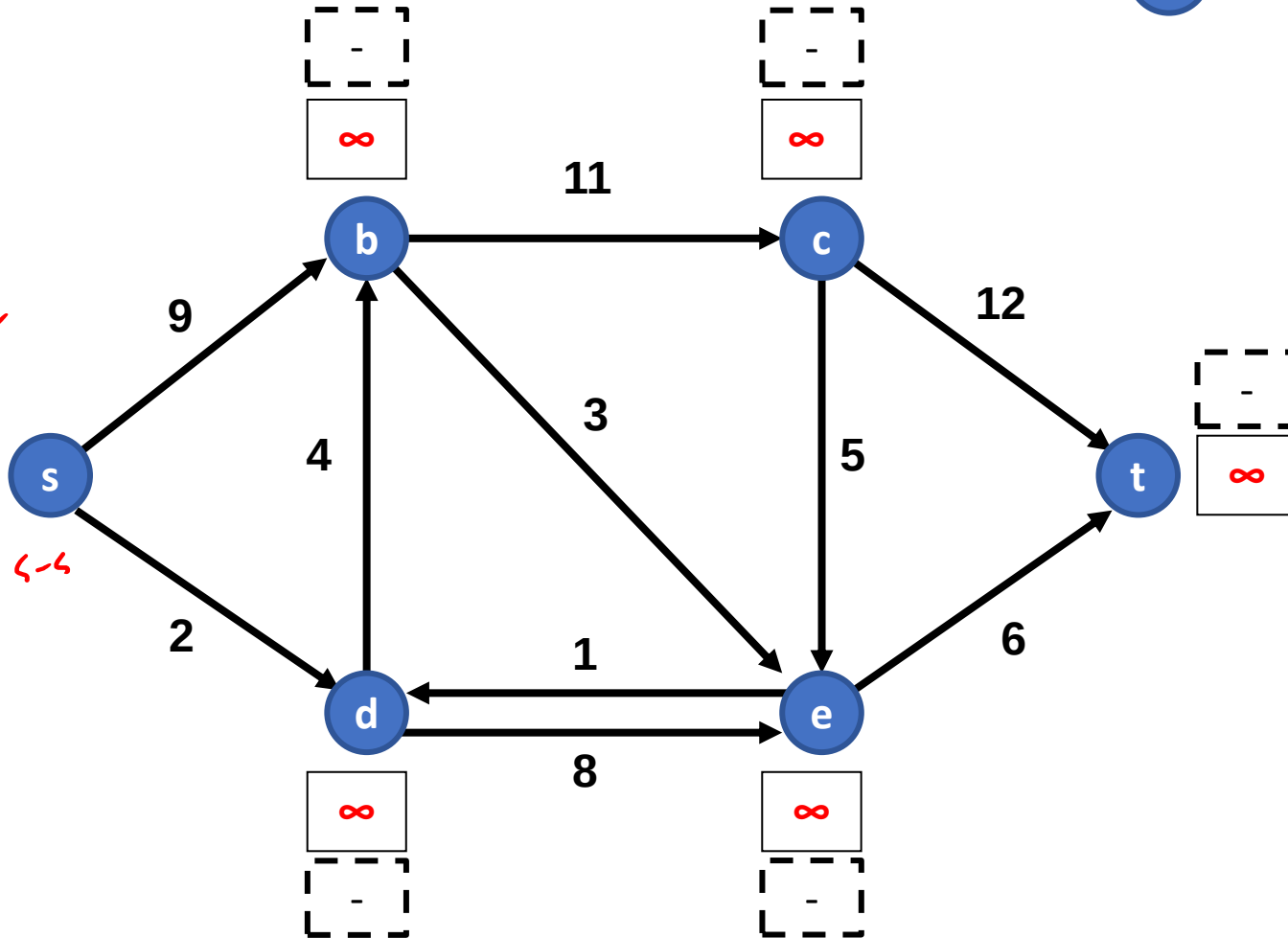
v Permanent
v Temporary

distance label
↓
Predecessor label
↑

v	Label	d_v	π_v
s	T	0	-
b	T		
c	T		
d	T		
e	T		
t	T		



s - t



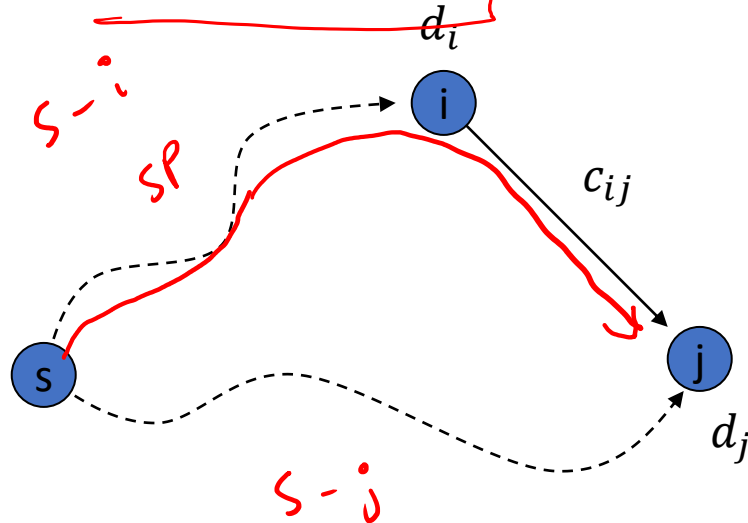
Table

Initialise the network, source vertex has a distance of 0 and all other vertices a distance of ∞

Improving Cost Labels

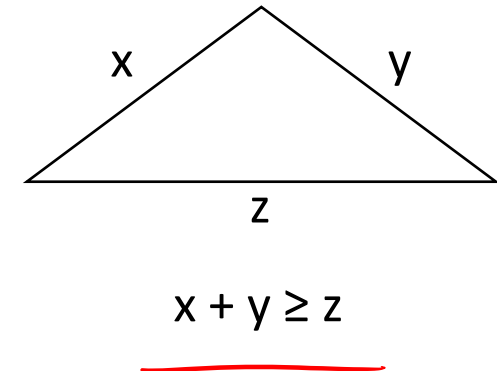
- Shortest path costs satisfy the triangle inequality:

$$d_i + c_{ij} \geq d_j \text{ for every edge } (i, j).$$



$$d_i + c_{ij} \geq d_j$$

$$d_j = d_i + c_{ij}$$

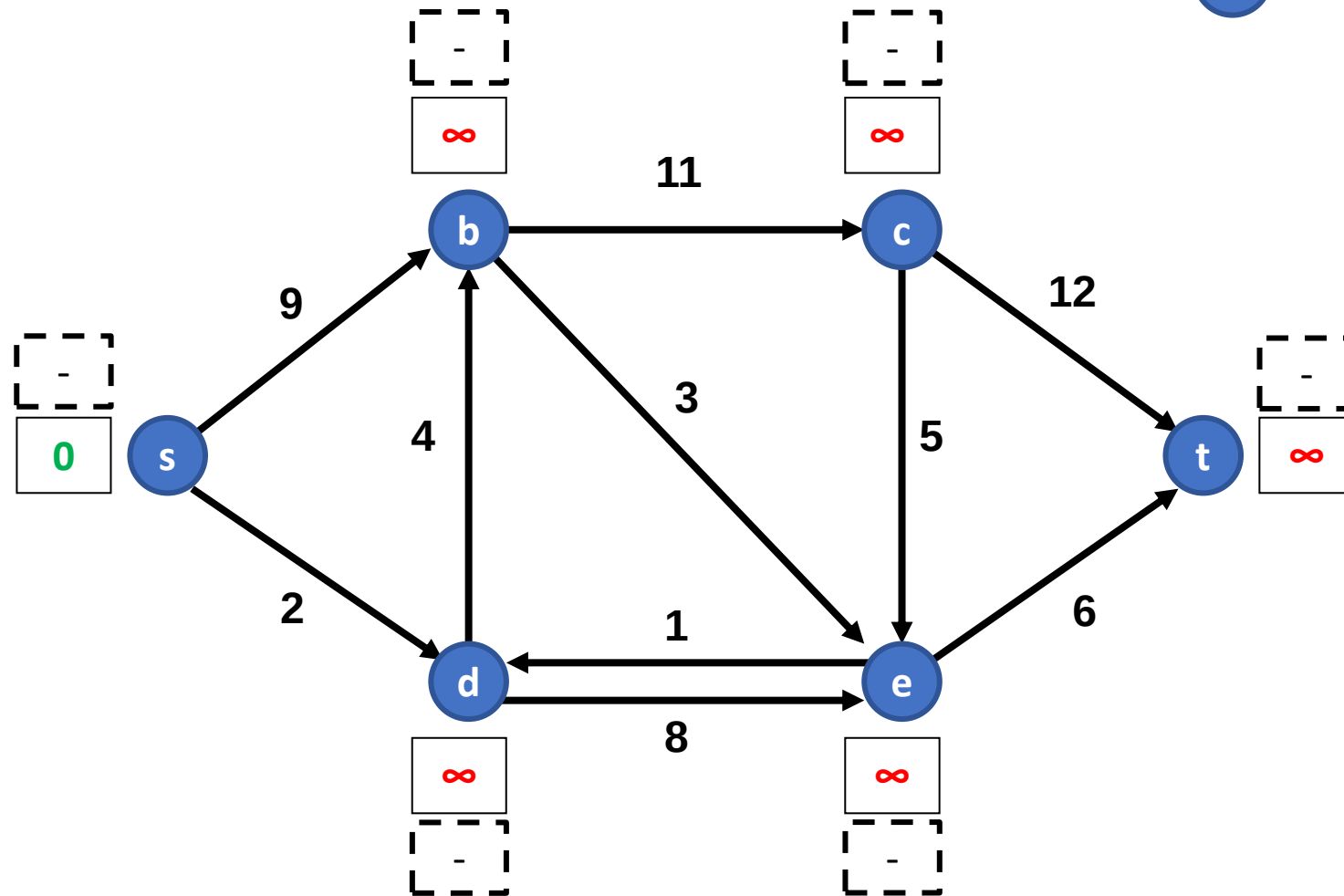


- If this is not **TRUE** then there exists some path that improves d_j
 - E.g. The path to d_i + the edge (i, j)

Dijkstra Example



v	Label	d_v	π_v
s	T	0	-
b	T		
c	T		
d	T		
e	T		
t	T		

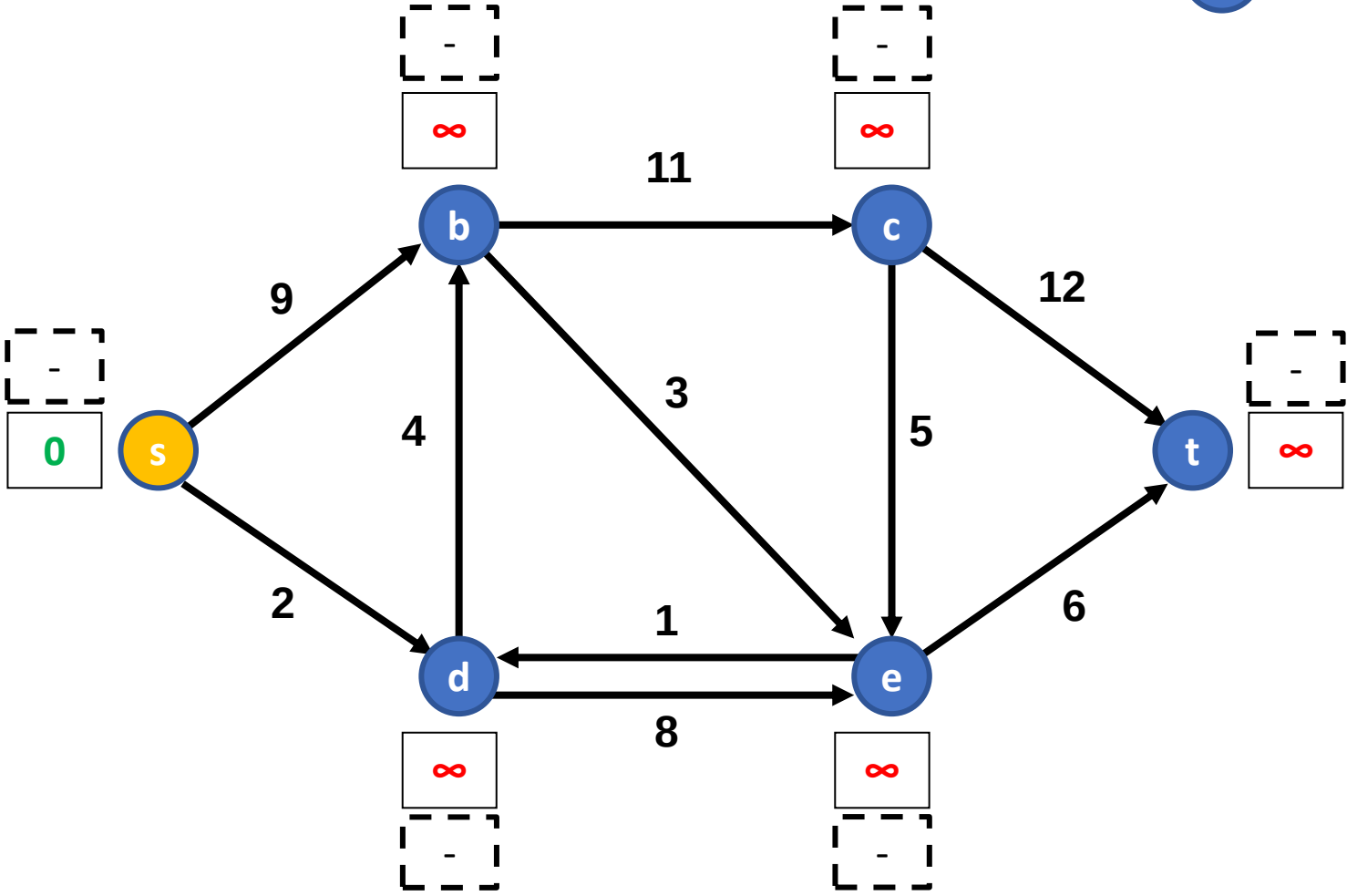


Initialise the network, source vertex has a distance of 0 and all other vertices a distance of ∞

Example



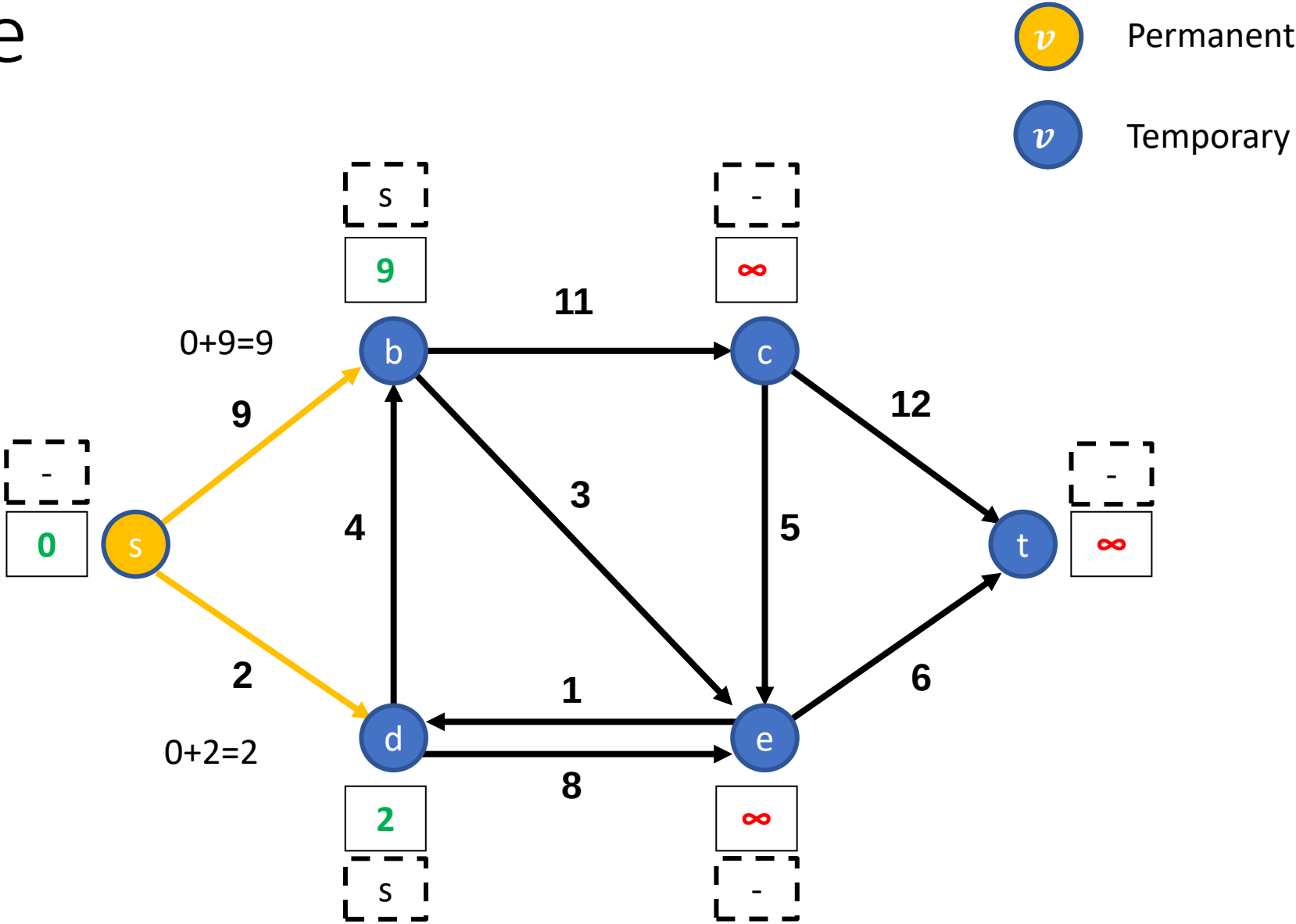
v	Label	d_v	π_v
s	P	0	-
b	T		
c	T		
d	T		
e	T		
t	T		



Label temporary vertex with minimum distance as permanent

Example

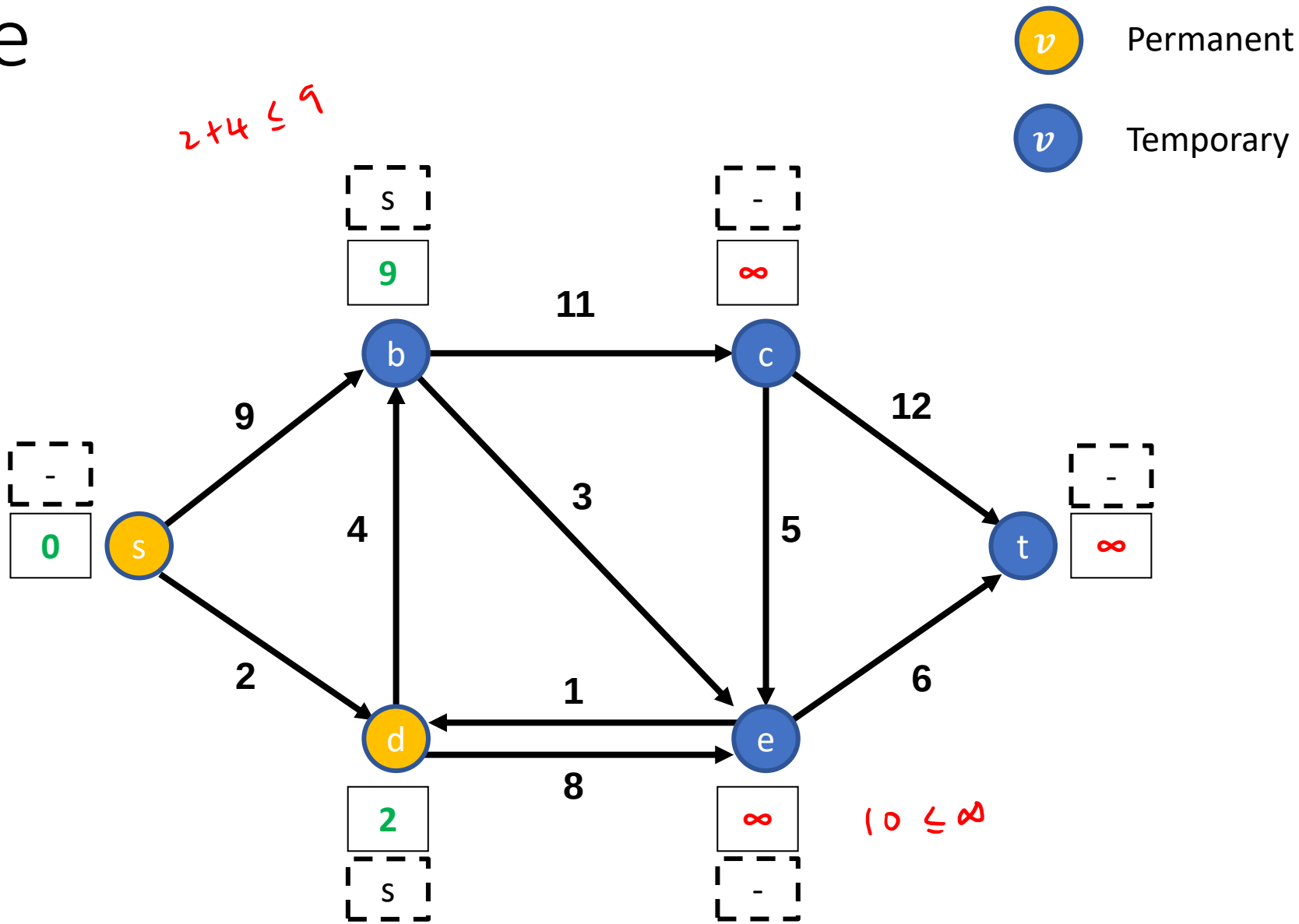
v	Label	d_v	π_v
s	P	0	-
b	T	9	s
c	T		
d	T	2	s
e	T		
t	T		



Update distances to reachable vertices which are labelled temporary

Example

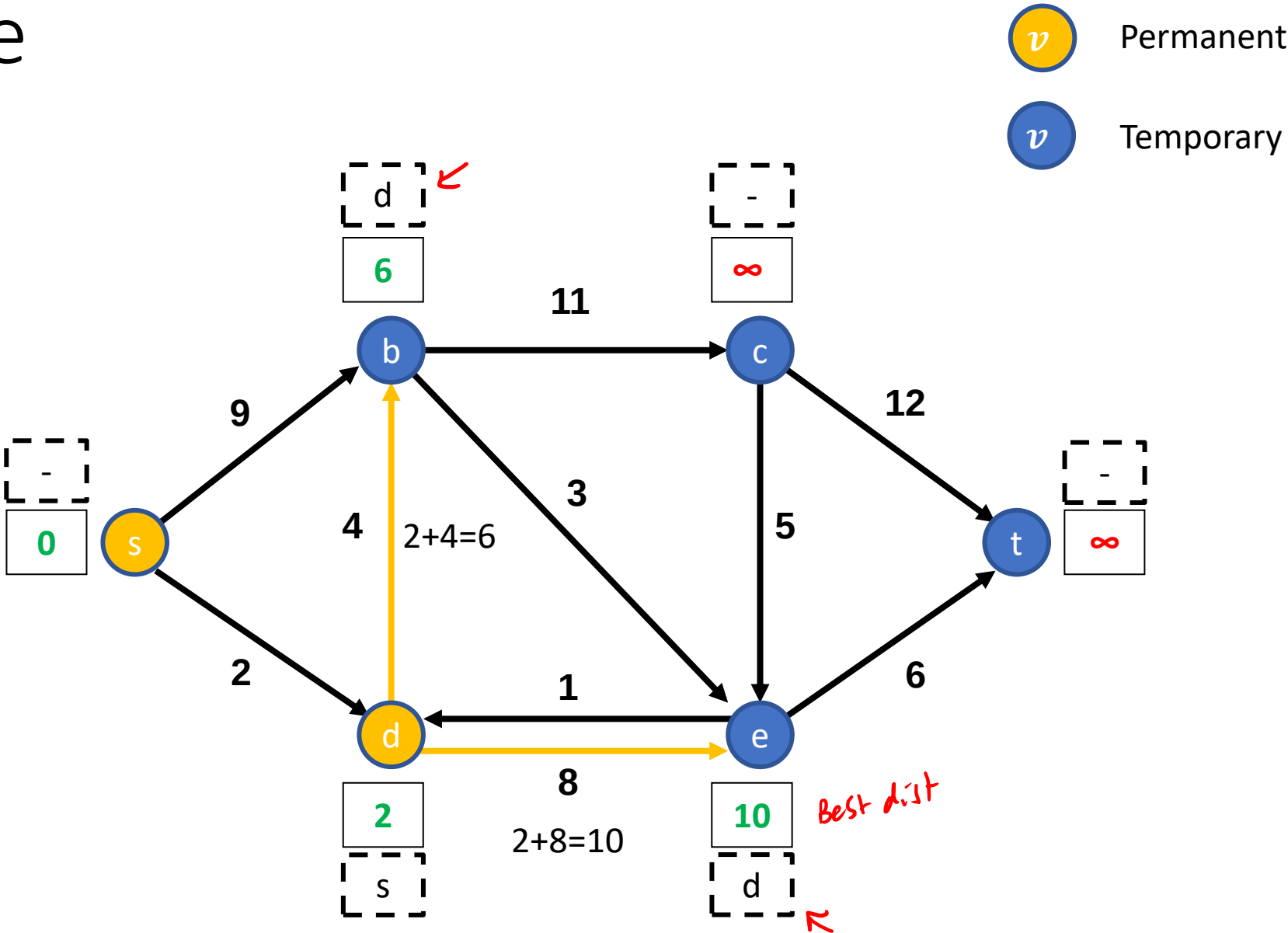
v	Label	d_v	π_v
s	P	0	-
b	T	9	s
c	T		
d	P	2	s
e	T		
t	T		



Label temporary vertex with minimum distance as permanent

Example

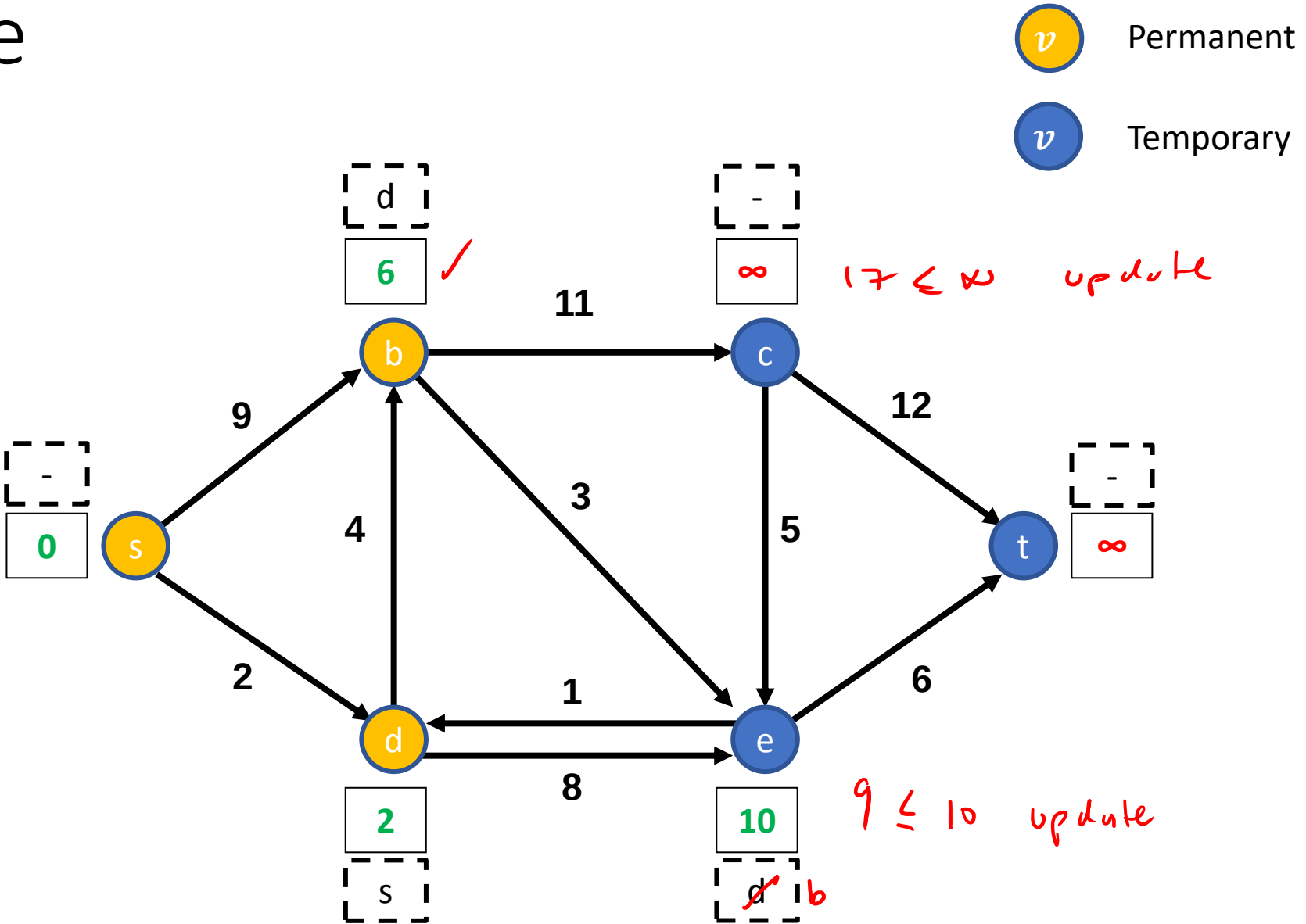
v	Label	d_v	π_v
s	P	0	-
b	T	6	s d
c	T		
d	P	2	s
e	T	10	d
t	T		



Update distances to reachable vertices which are labelled temporary

Example

v	Label	d_v	π_v
s	P	0	-
b	P	9	s d
c	T		
d	P	2	s
e	T	10	d
t	T		

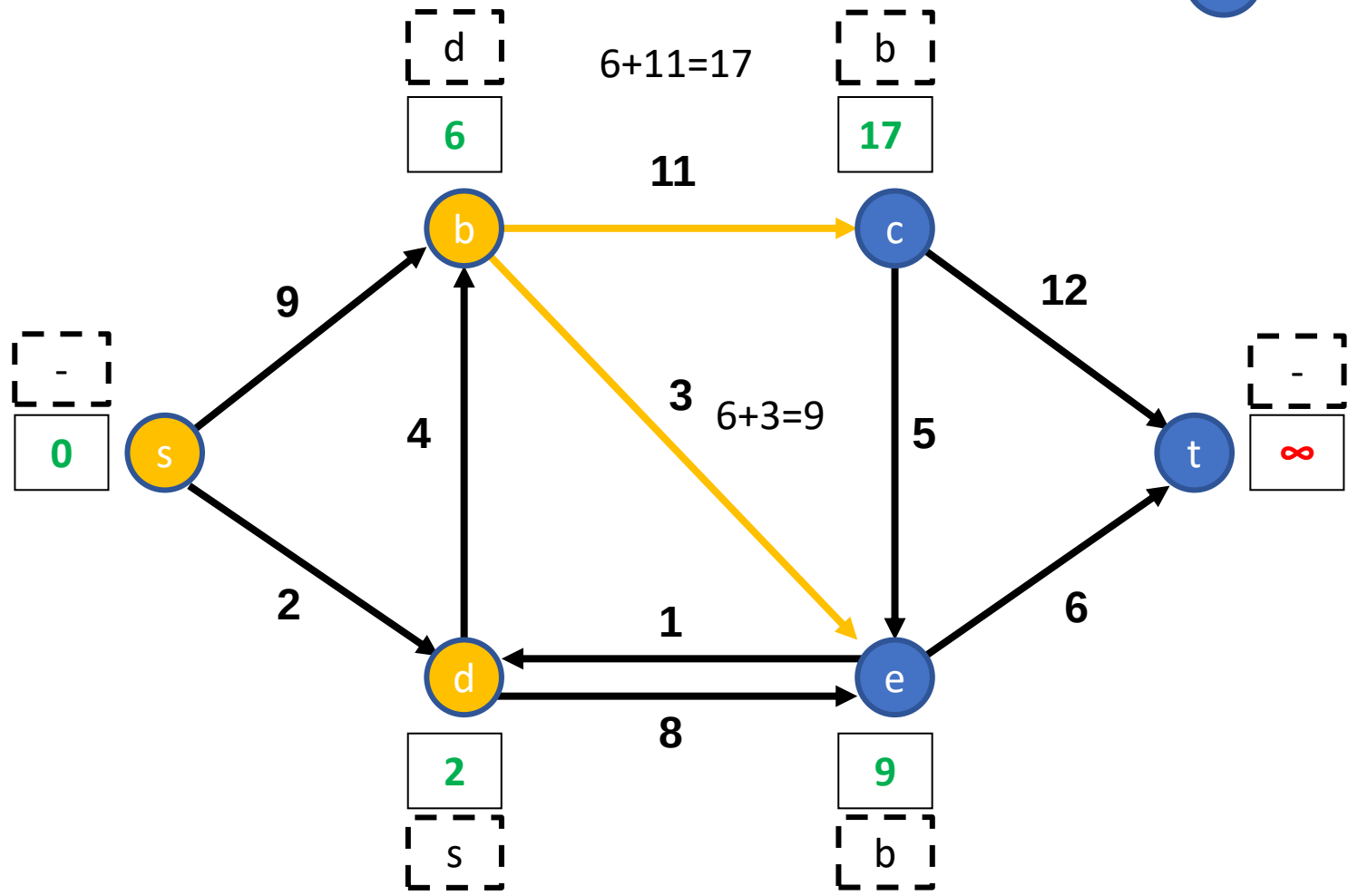


Label temporary vertex with minimum distance as permanent

Example



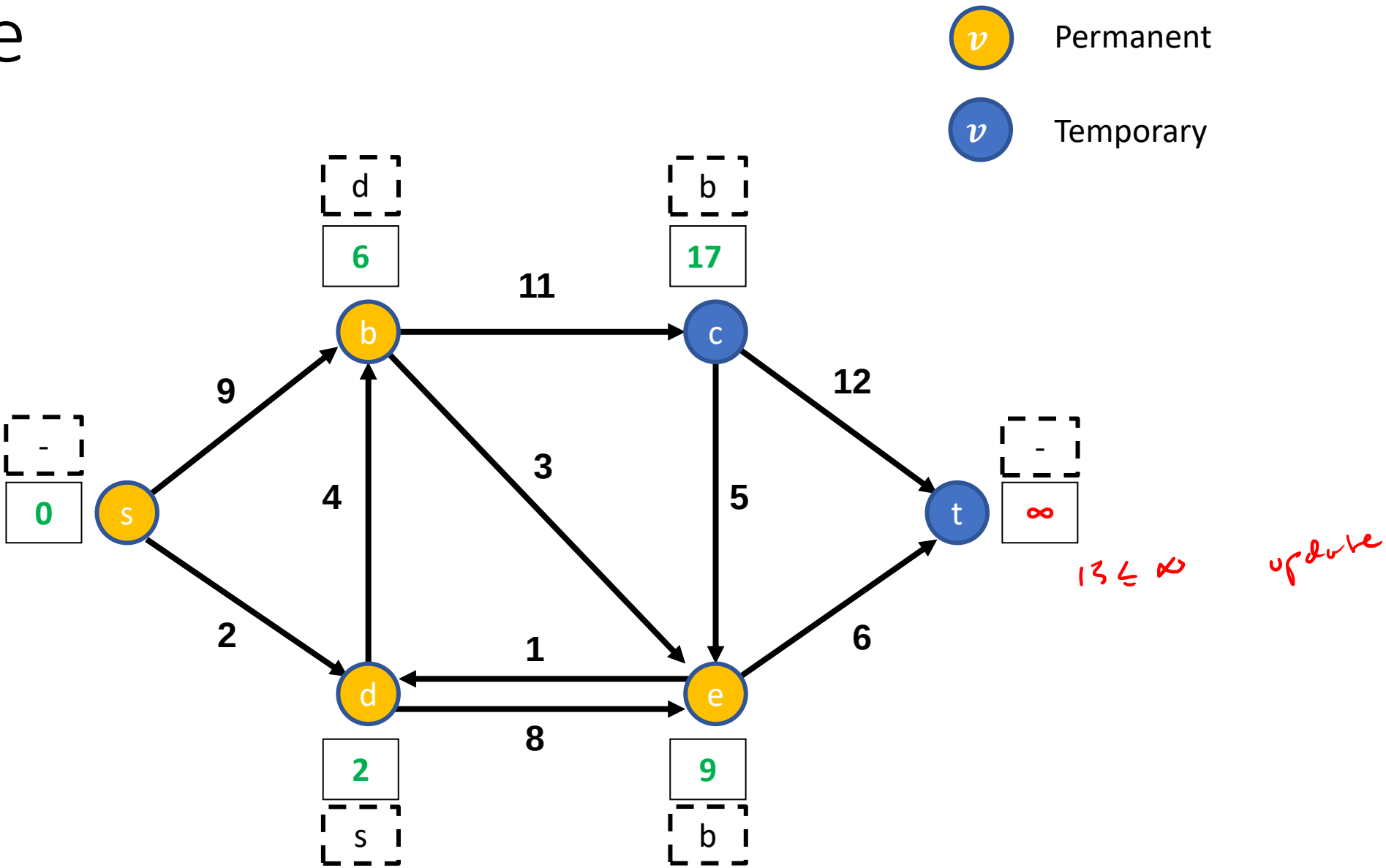
v	Label	d_v	π_v
s	P	0	-
b	P	6	s
c	T	17	b
d	P	2	s
e	T	9	d
t	T		



Update distances to reachable vertices which are labelled temporary

Example

v	Label	d_v	π_v
s	P	0	-
b	P	9	s d
c	T	17	b
d	P	2	s
e	P	9	d b
t	T		

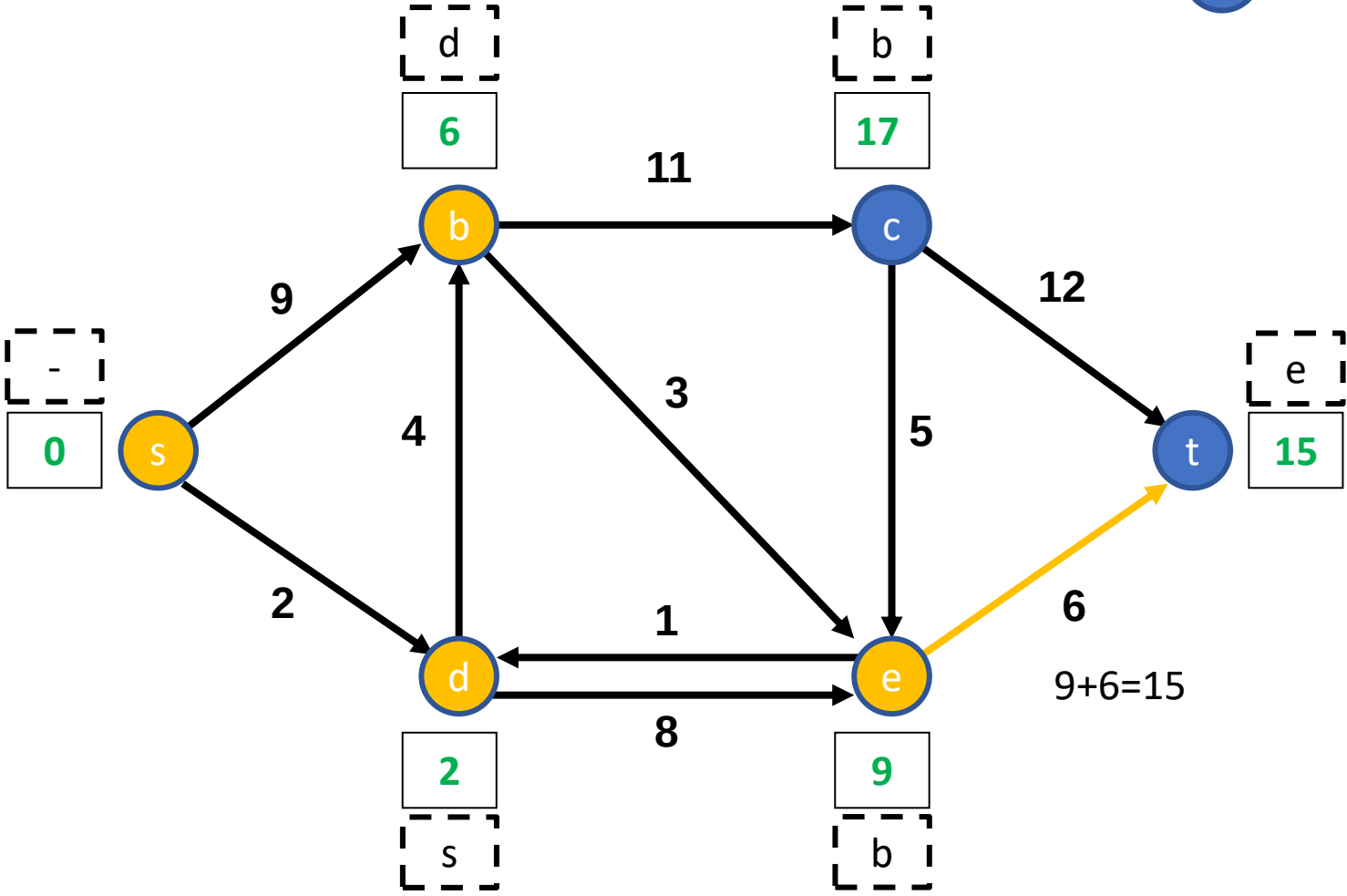


Label temporary vertex with minimum distance as permanent

Example



v	Label	d_v	π_v
s	P	0	-
b	P	9	s
c	T	17	b
d	P	2	s
e	P	9	d
t	T	15	e

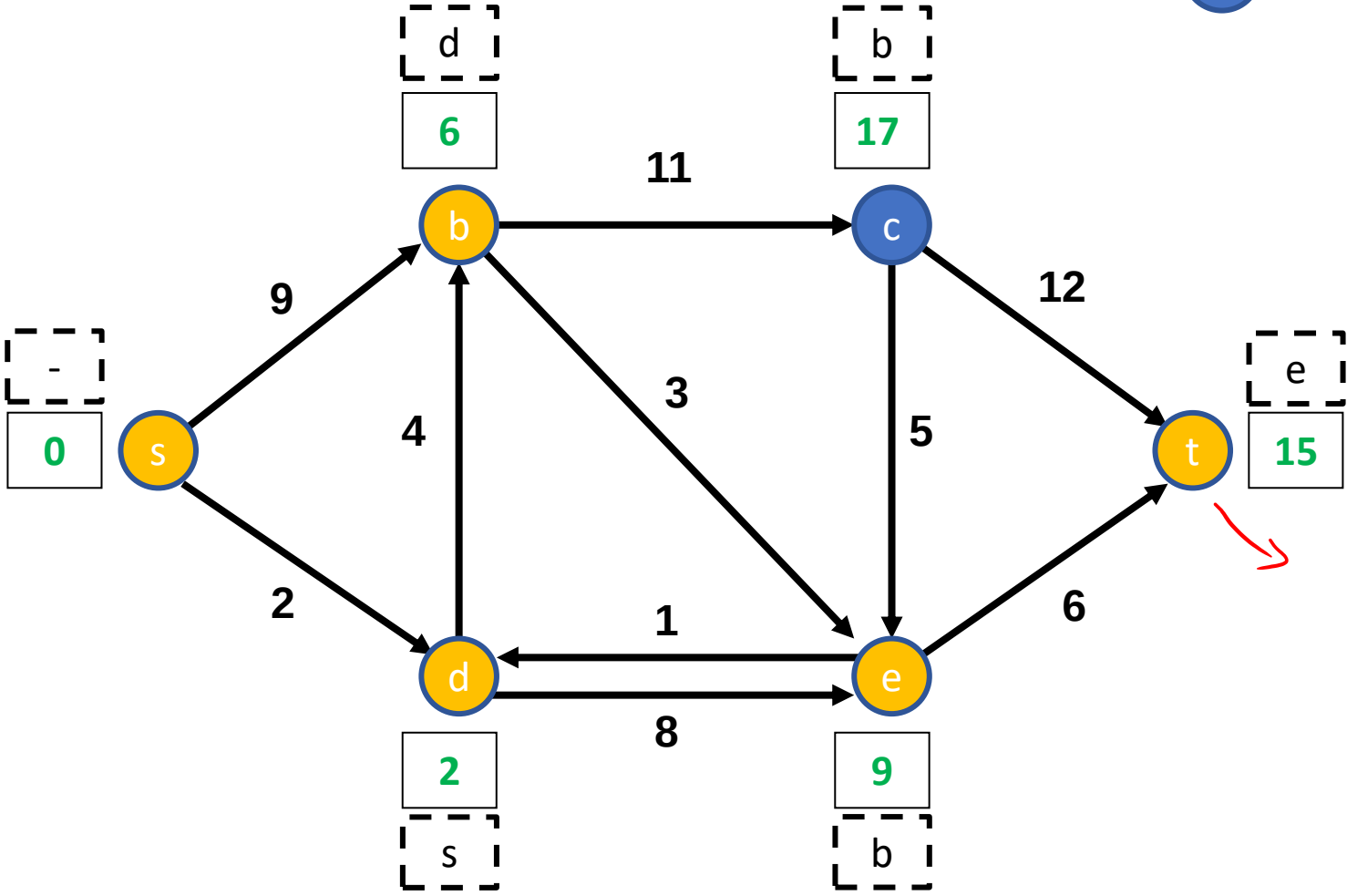


Update distances to reachable vertices which are labelled temporary

Example



v	Label	d_v	π_v
s	P	0	-
b	P	9	s d
c	T	17	b
d	P	2	s
e	P	10	d b
t	P	15	e

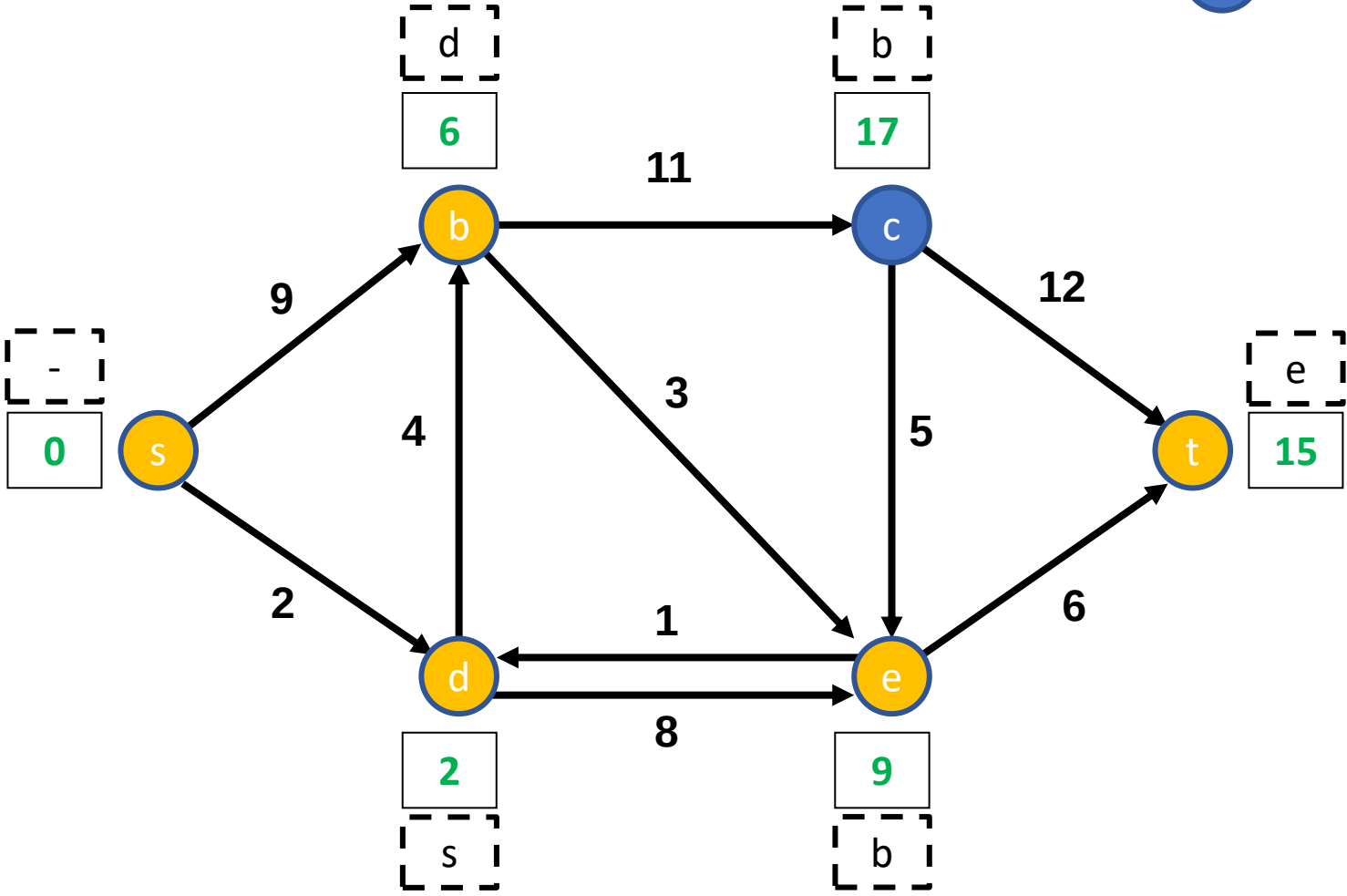


Label temporary vertex with minimum distance as permanent

Example



v	Label	d_v	π_v
s	P	0	-
b	P	9	s d
c	T	17	b
d	P	2	s
e	P	10	d b
t	P	15	e



Update distances to reachable vertices which are labelled temporary. No changes

Example

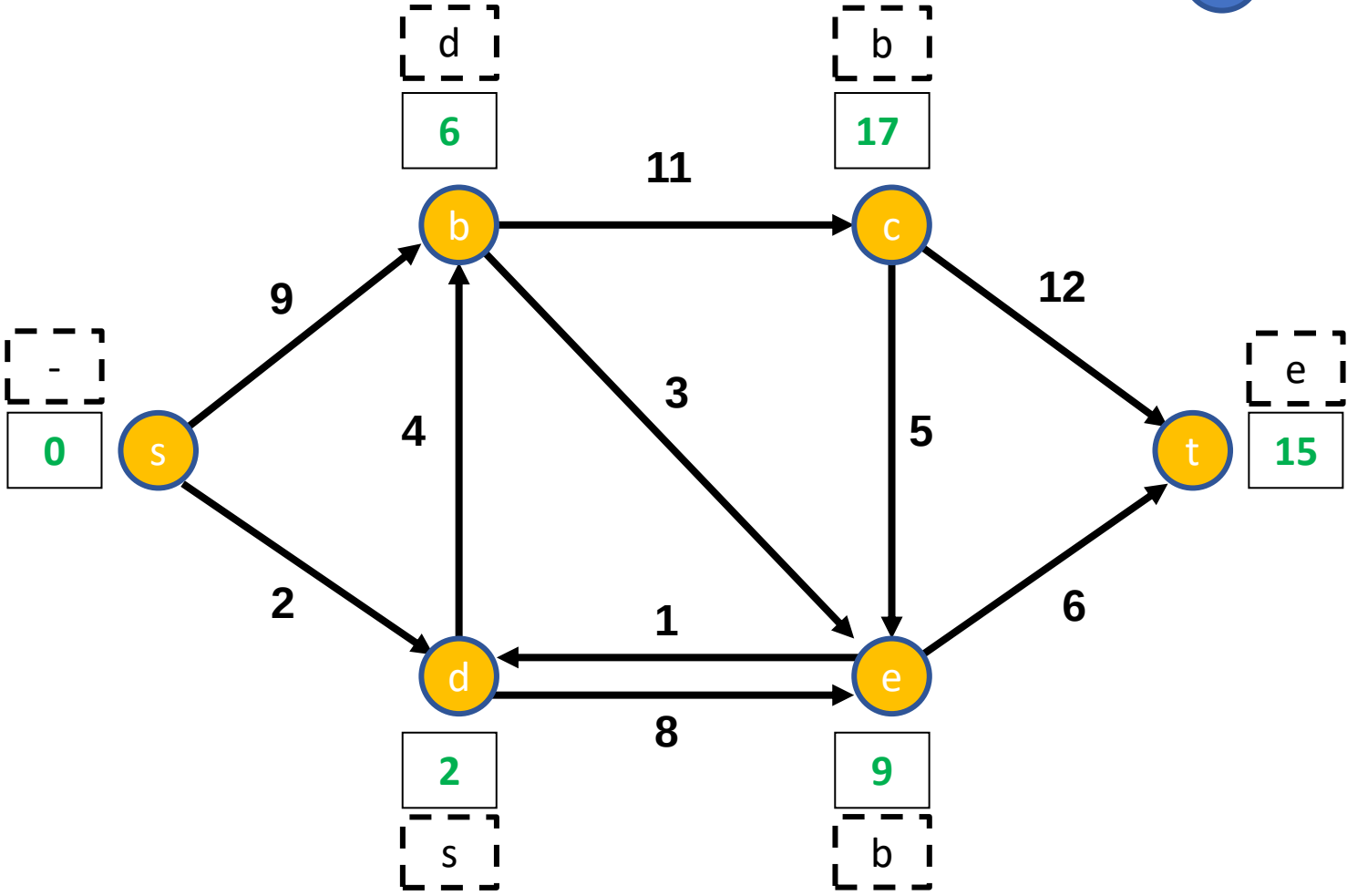


Permanent



Temporary

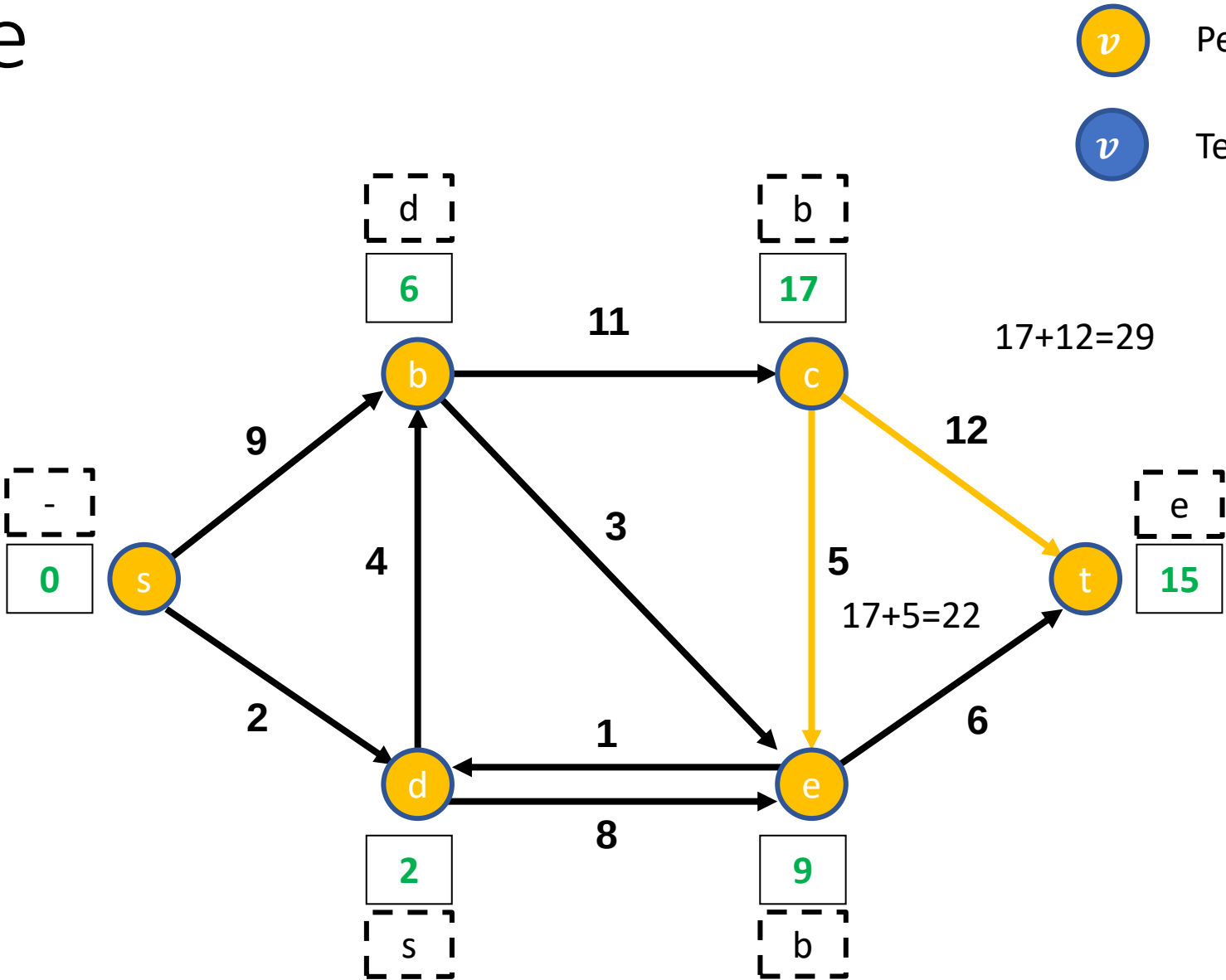
v	Label	d_v	π_v
s	P	0	-
b	P	9	s d
c	P	17	b
d	P	2	s
e	P	10	d b
t	P	15	e



Label temporary vertex with minimum distance as permanent

Example

v	Label	d_v	π_v
s	P	0	-
b	P	9	s
c	P	17	b
d	P	2	s
e	P	10	d
t	P	15	e

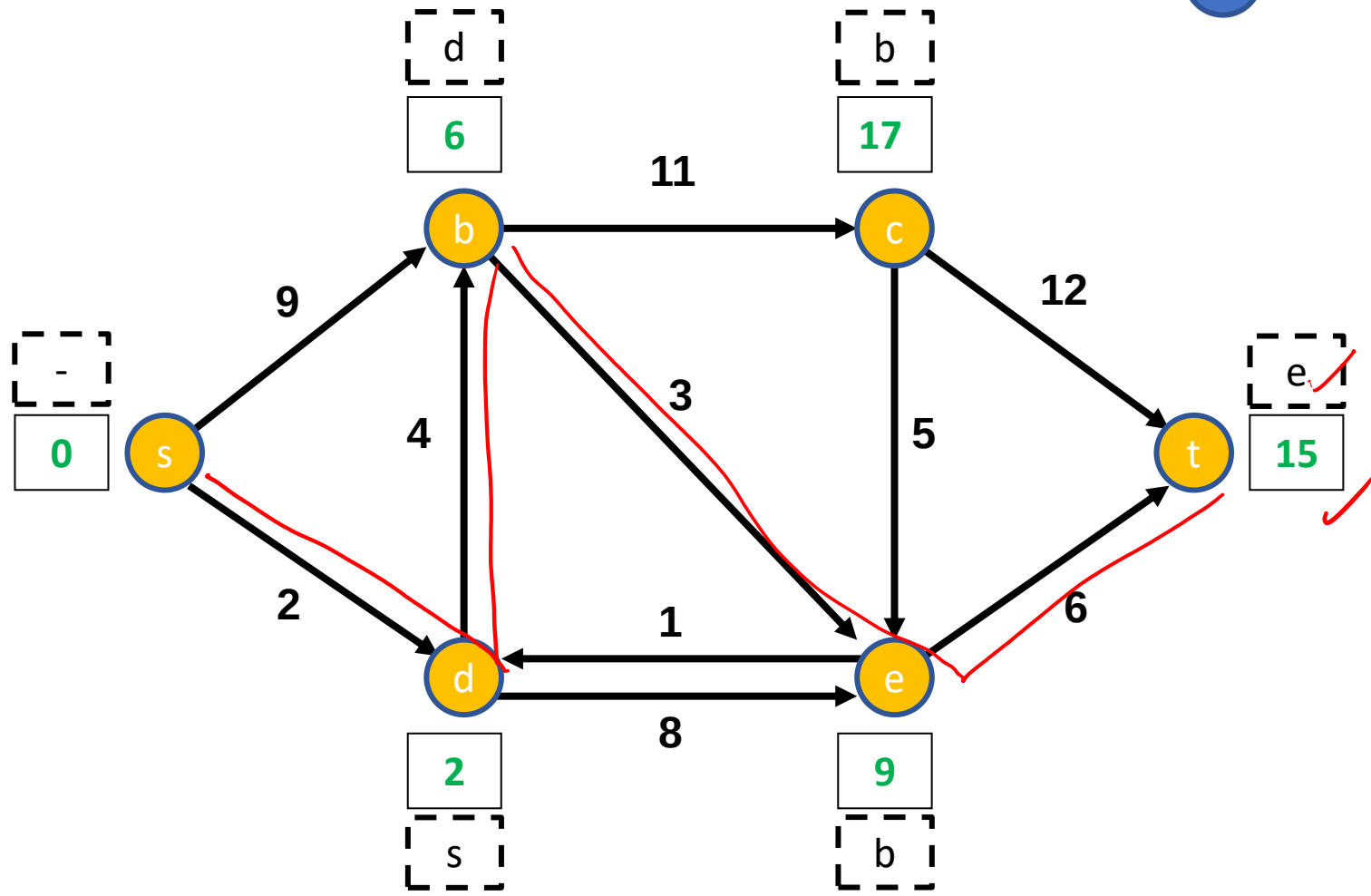


Update distances to reachable vertices which are labelled temporary. No changes

Example



v	Label	d_v	π_v
s	P	0	-
b	P	9	s d
c	P	17	b
d	P	2	s
e	P	10	d b
t	P	15	e

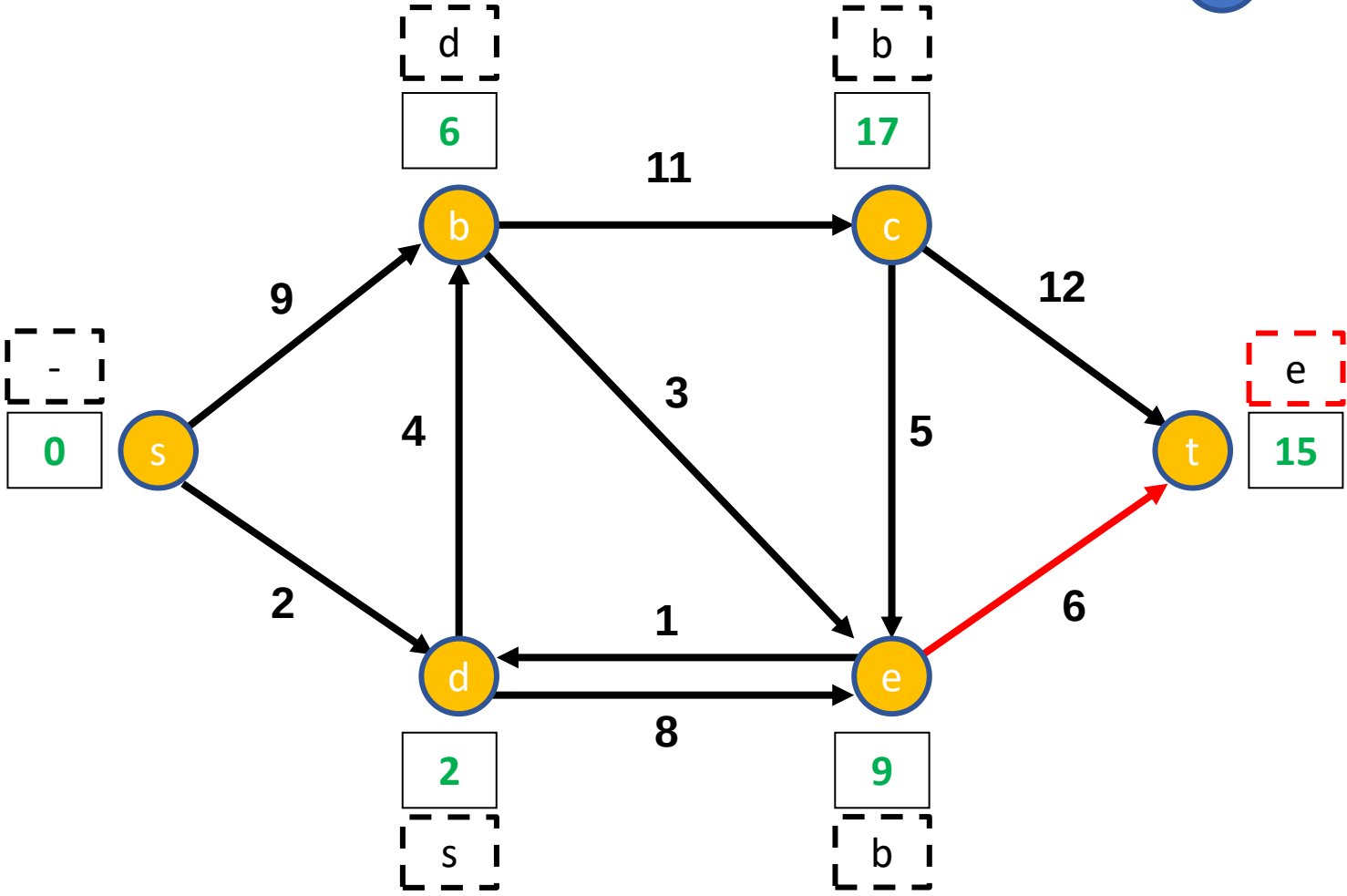


Final Network and Table

Example



v	Label	d_v	π_v
s	P	0	-
b	P	9	s d
c	P	17	b
d	P	2	s
e	P	10	d b
t	P	15	e

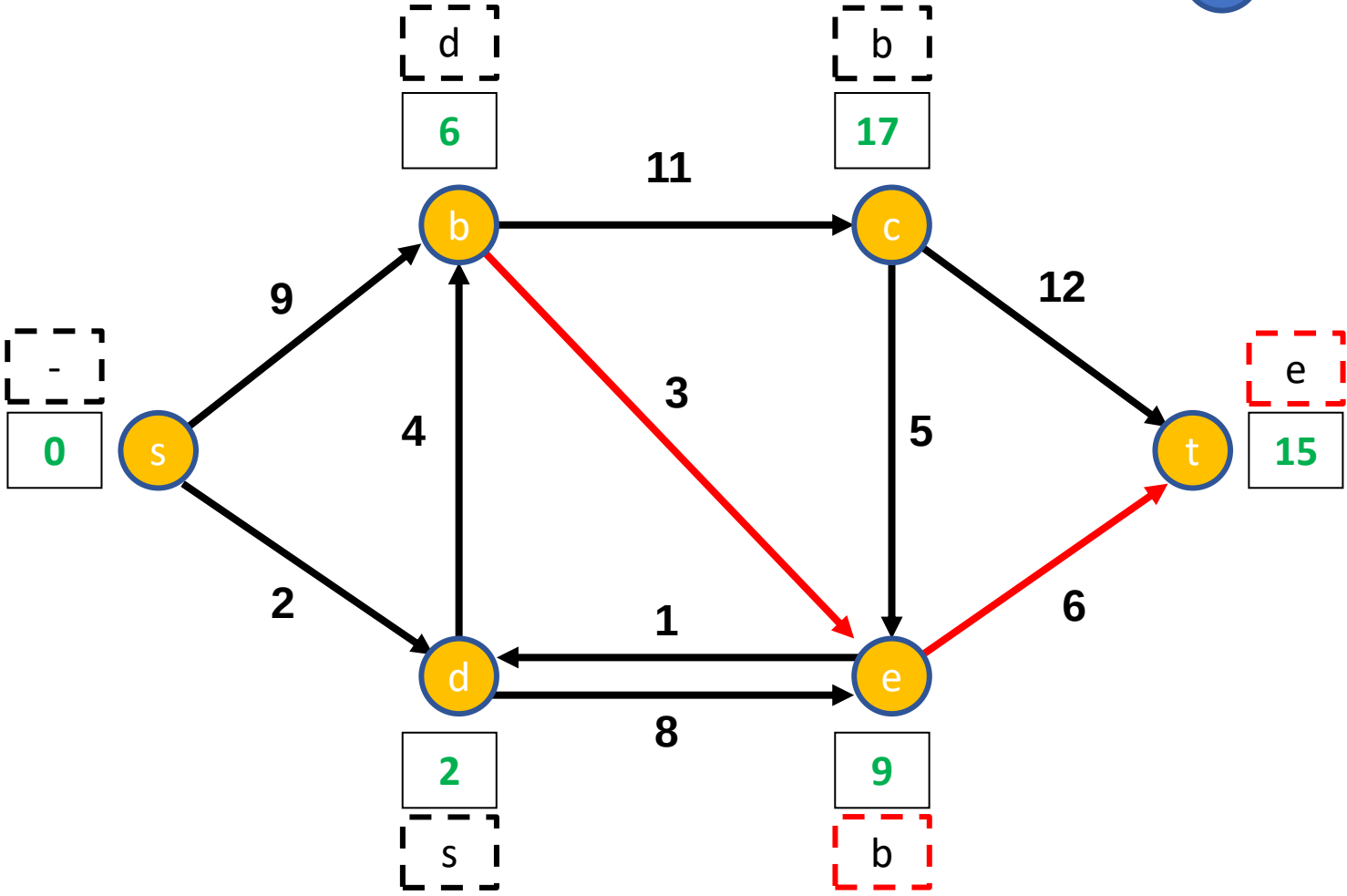


To compute the shortest path, work backwards from the source along the predecessor

Example



v	Label	d_v	π_v
s	P	0	-
b	P	9	s d
c	P	17	b
d	P	2	s
e	P	10	d b
t	P	15	e

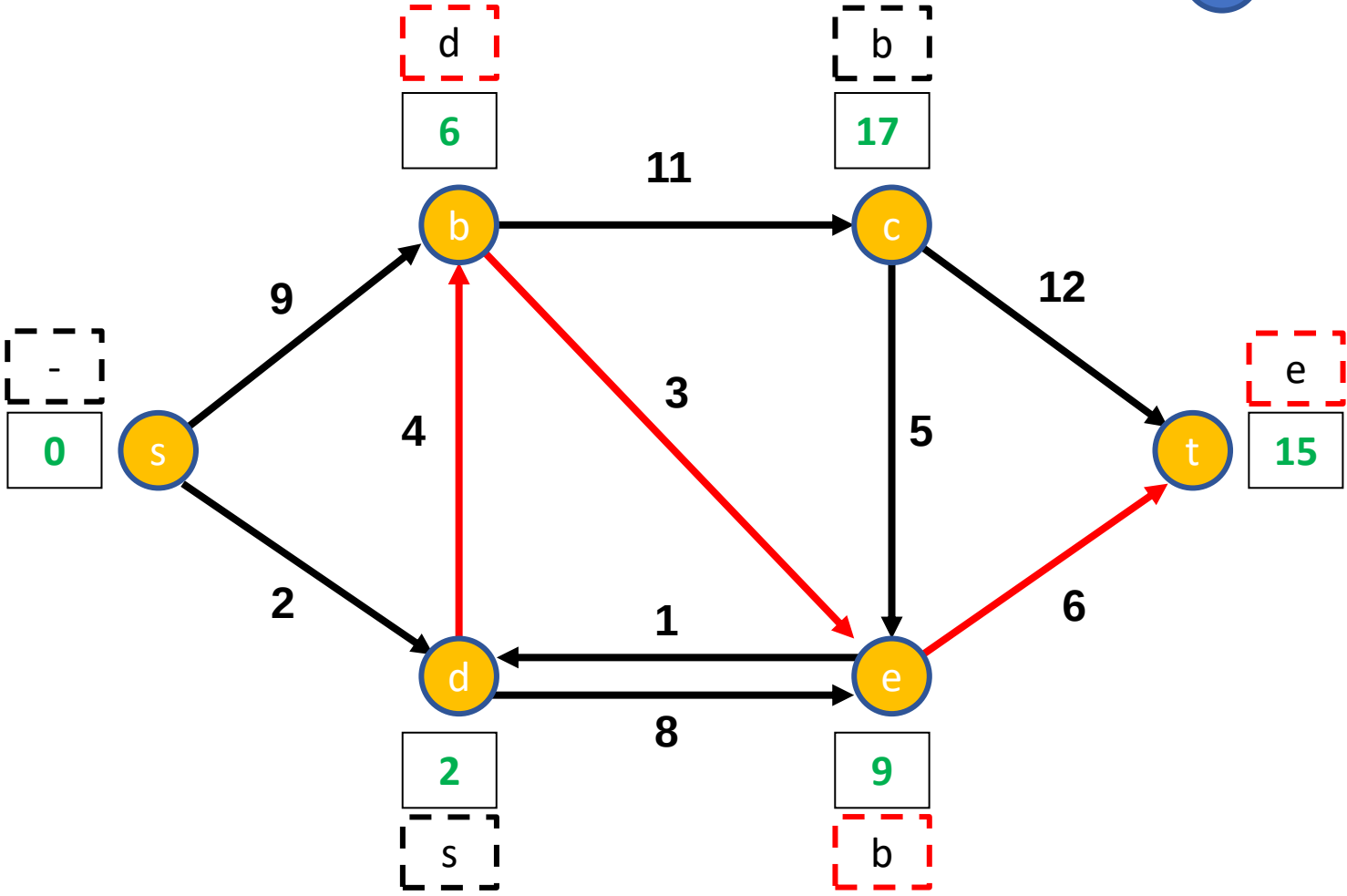


To compute the shortest path, work backwards from the source along the predecessor

Example



v	Label	d_v	π_v
s	P	0	-
b	P	6	s d
c	P	17	b
d	P	2	s
e	P	9	d b
t	P	15	e

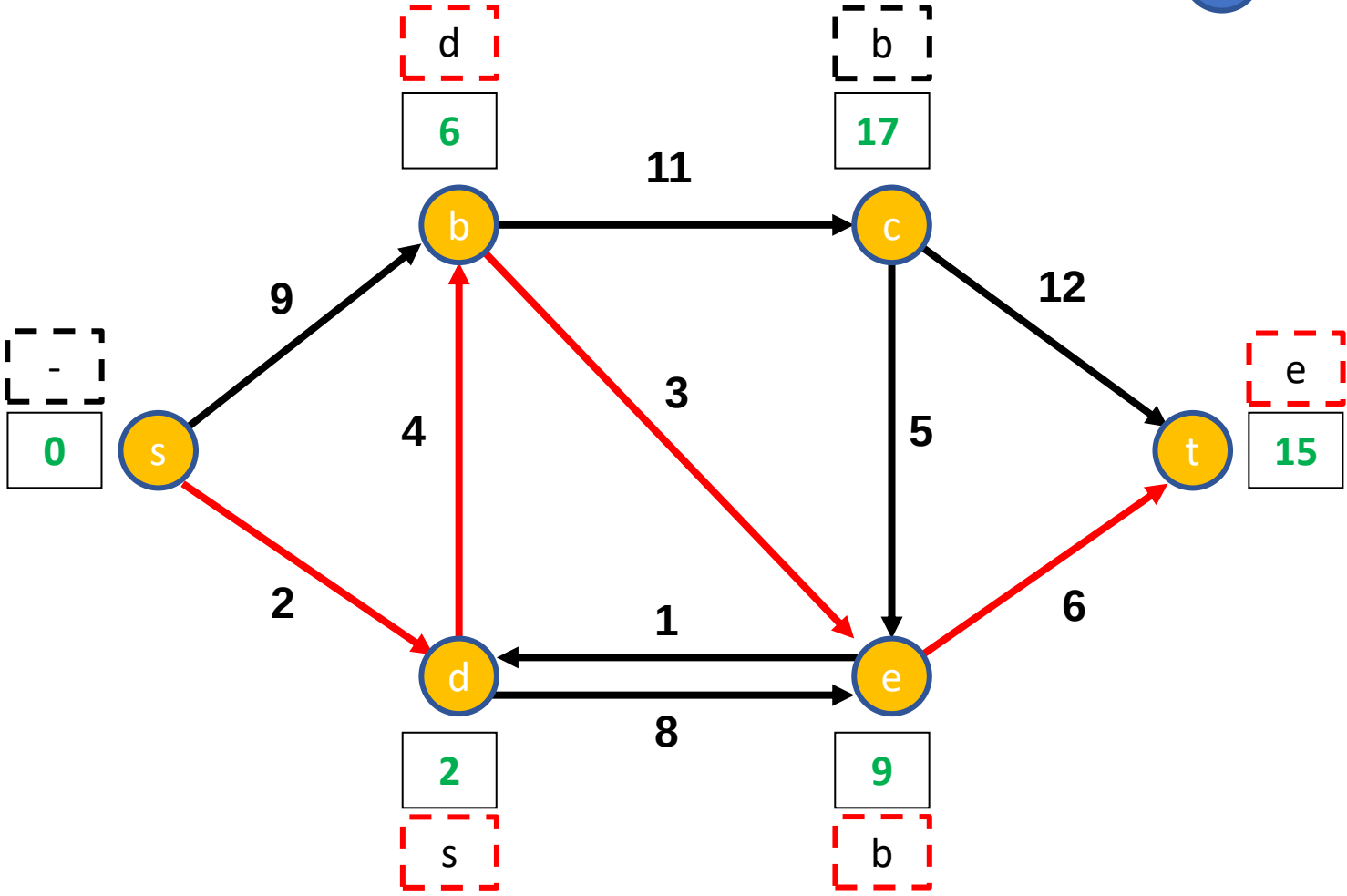


To compute the shortest path, work backwards from the source along the predecessor

Example



v	Label	d_v	π_v
s	P	0	-
b	P	9	s d
c	P	17	b
d	P	2	s
e	P	10	d b
t	P	15	e

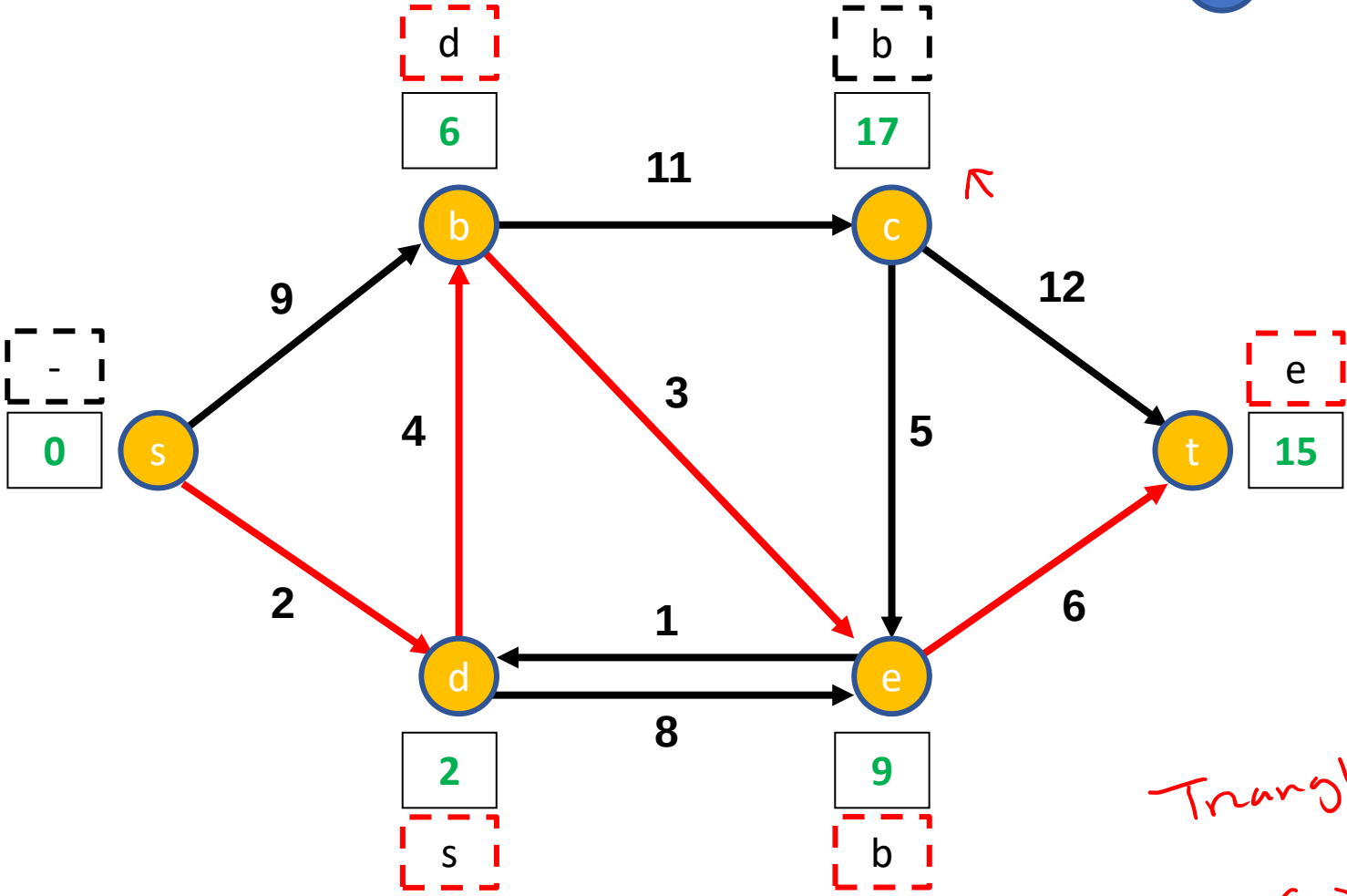


To compute the shortest path, work backwards from the source along the predecessor

Example



v	Label	d_v	π_v
s	P	0	-
b	P	9	s
c	P	17	b
d	P	2	s
e	P	10	d
t	P	15	e



Shortest path from s to t is $s - d - b - e - t$ with a path length of 15

Correctness

$a + nd$

$x + y$

$p + r$

Define your notation

Use notation that is
used by others

✓ Visits all vertices in increasing order of distance and visits each vertex exactly once.

✓ Dijkstra maintains a set of visited vertices (permanent) and unvisited vertices (temporary).

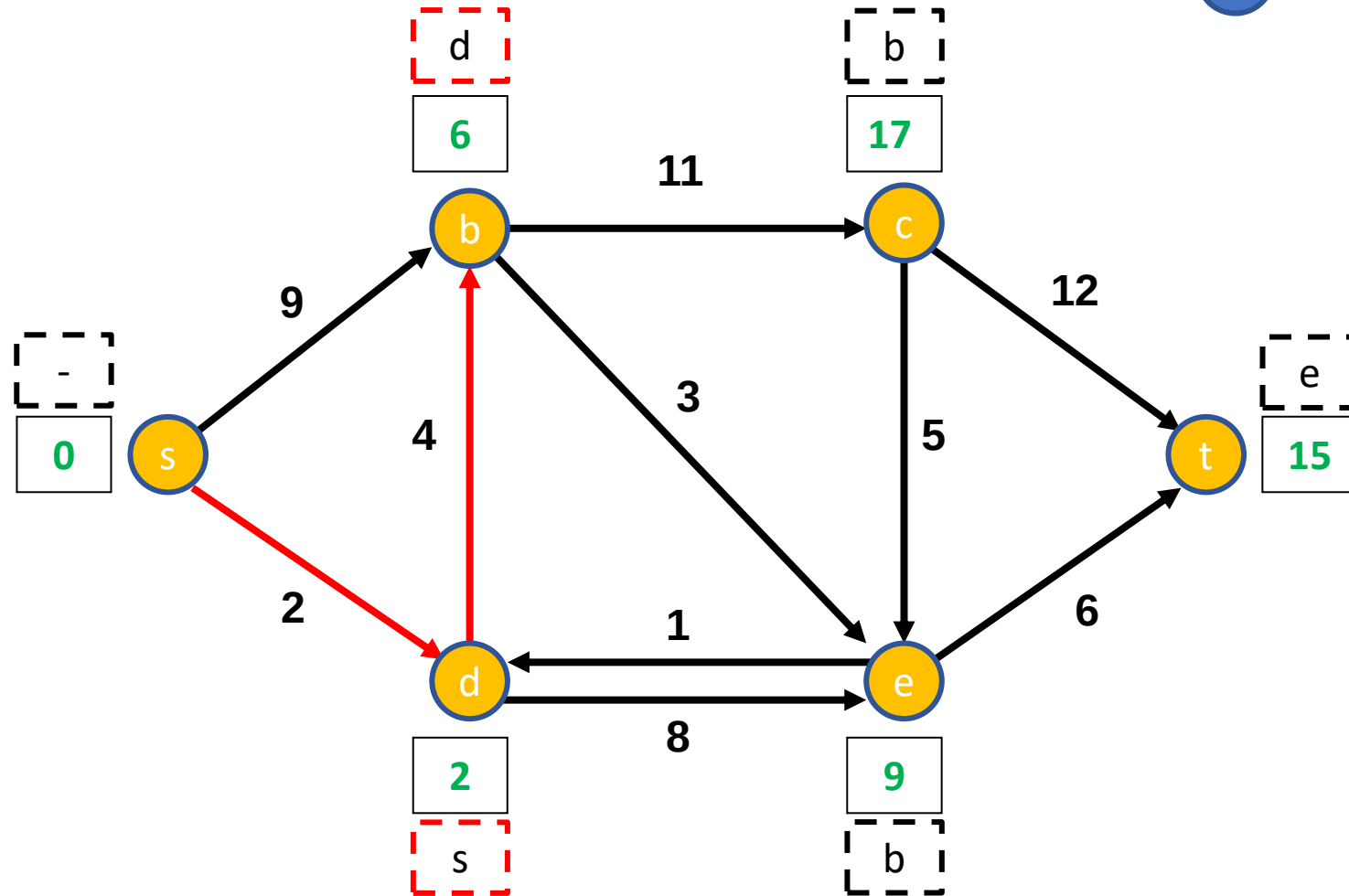
Claim: For a visited vertex v , d_v and π_v will be the optimal shortest path distance and corresponding predecessor vertex.

| Meaning, we get all the shortest paths from the source to all visited vertices free! |

Shortest sub-path



v	Label	d_v	π_v
s	P	0	-
b	P	6	s d
c	P	17	b
d	P	2	s
e	P	9	d b
t	P	15	e



For example the shortest path from vertex s to b is $s - d - b$ of length 6

Proof and Time Complexity

Dijkstra's Algorithm has a worse case time complexity of

$O(V^2)$

- $O(|V| \log |V|)$ with an adjacency list
- $O((|E| + |V|) \log |V|)$ with a binary heap

← Data structures

Fibonacci heap

$V \log V < V^2$ $100^2 <<< 100!$

$O(V!)$

Running Time

Linear

Exponential

If you are interested you can find all the proofs about the correctness and time complexities of Dijkstra's and the other algorithms in:

✓ Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. (2001). *Introduction to Algorithms*. The MIT Press. ISBN: 0262032937

Or check out

Geek & geeks

Note they are somewhat complicated if you haven't seen this type of argument before.

30 seconds....

You have 30 seconds to find wally.

Raise your hand if you find him and I will pause to check.



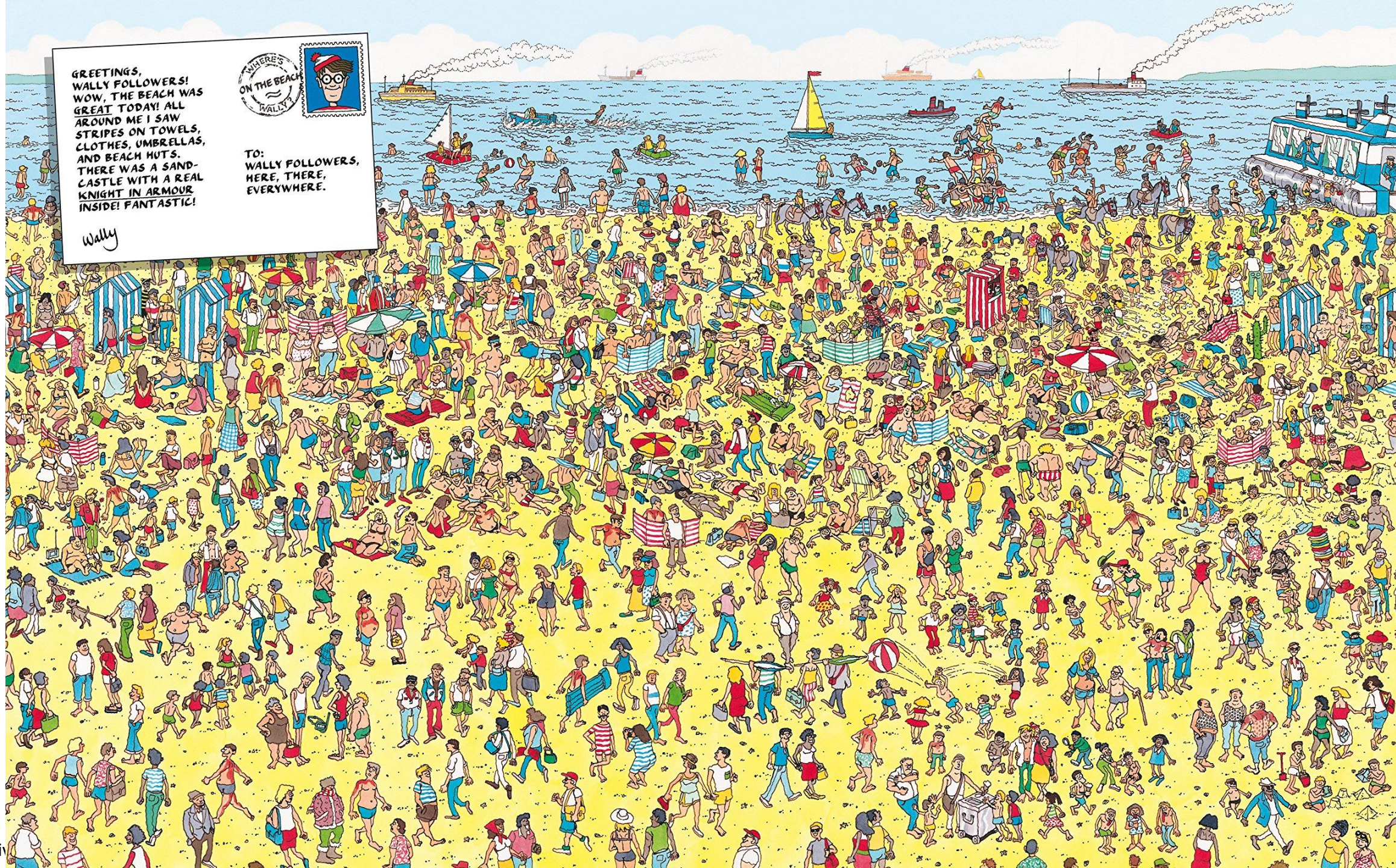
Winning answer receives a chocolate bar, no winning answers will roll over to next week.

GREETINGS,
WALLY FOLLOWERS!
WOW, THE BEACH WAS
GREAT TODAY! ALL
AROUND ME I SAW
STRIPES ON TOWELS,
CLOTHES, UMBRELLAS,
AND BEACH HUTS.
THERE WAS A SAND-
CASTLE WITH A REAL
KNIGHT IN ARMOUR
INSIDE! FANTASTIC!

Wally

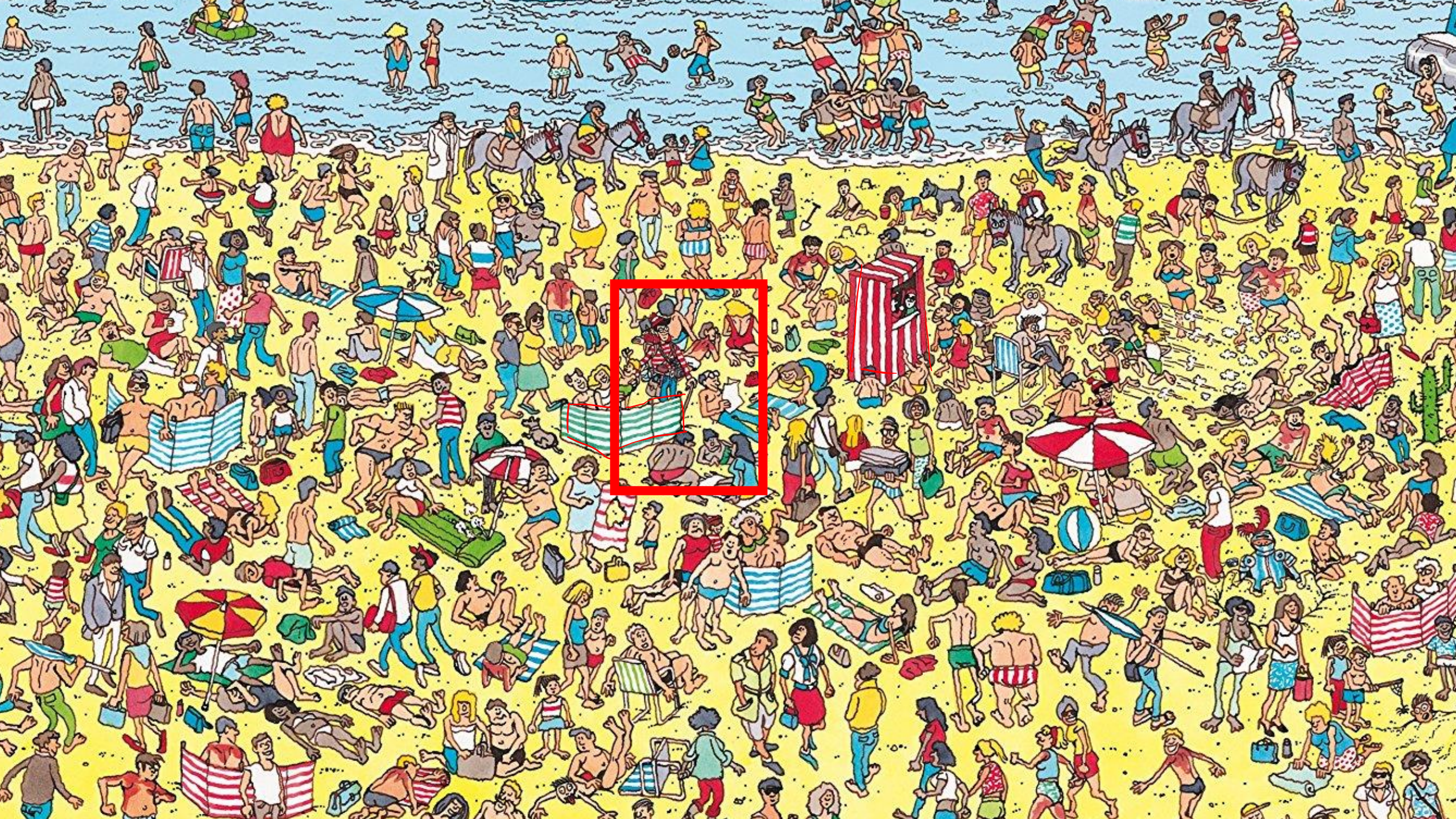


TO:
WALLY FOLLOWERS,
HERE, THERE,
EVERYWHERE.



Checking Answer....

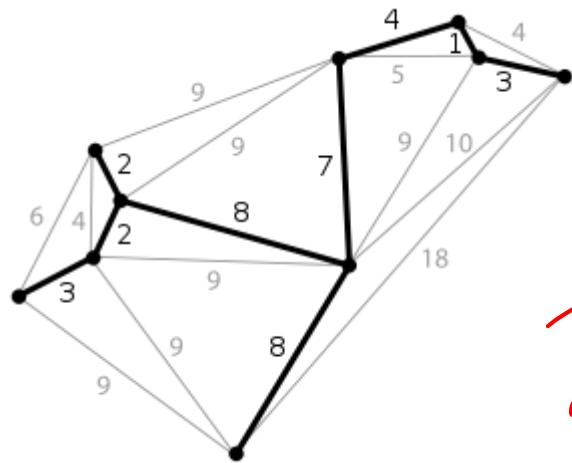
Please be patience, there some high tech stuff going on right now...



Minimum Spanning Tree

Definition

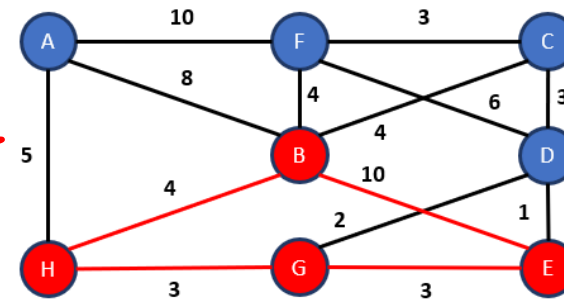
A subset of weighted edges that connects all the vertices of an weighted undirected graph together without any cycles and has minimum edge weight.



MST

*Tree (no cycles)
Spans all vertices
Minimum weight*

Minimum Spanning Tree



This is a cycle

Example Application

A fibre optics company is looking to expand it's portfolio by providing broadband to an entirely new set of areas. Currently it has cable laid to a particular location.

The cable is extremely expensive and particularly difficult to lay in certain areas, therefore the company wished to minimise the cost of laying the cable, whilst connected all areas.

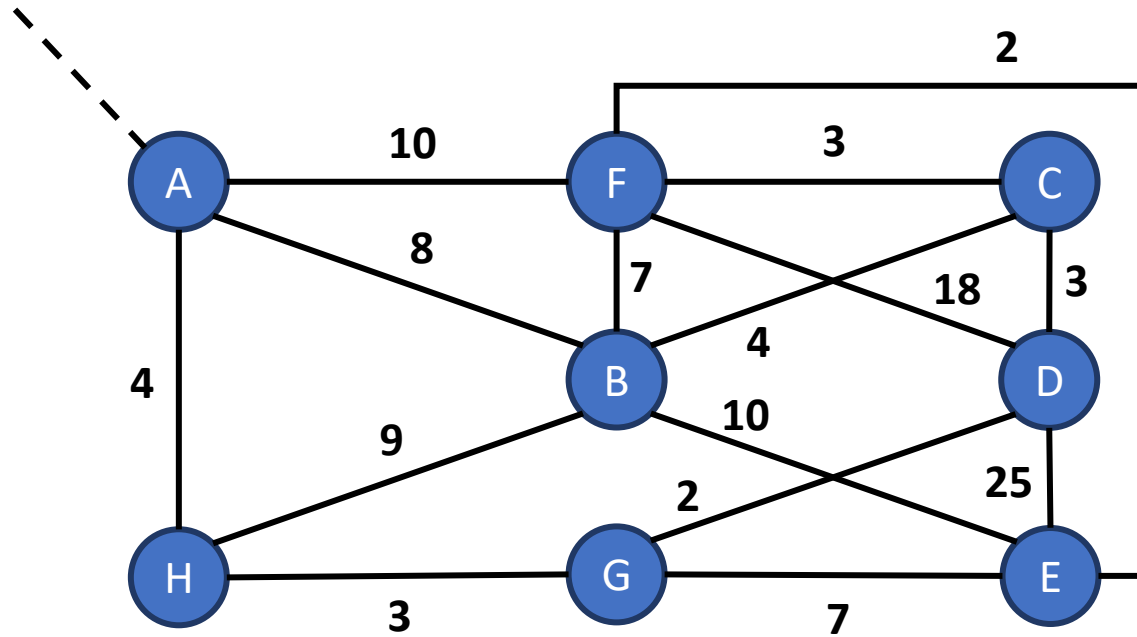
The company can lay cable between certain areas and has approximated the cost. Compute the minimum cost for connected^{ing} all the areas.

Prim's Algorithm

1. Initialise distances to vertices as ∞ and predecessors as —
2. Arbitrarily select a vertex and label as selected
3. Update distances and predecessors to adjacent unselected vertices
4. Select the reachable vertex with minimum distance
5. Repeat 3 and 4 until all vertices have been selected

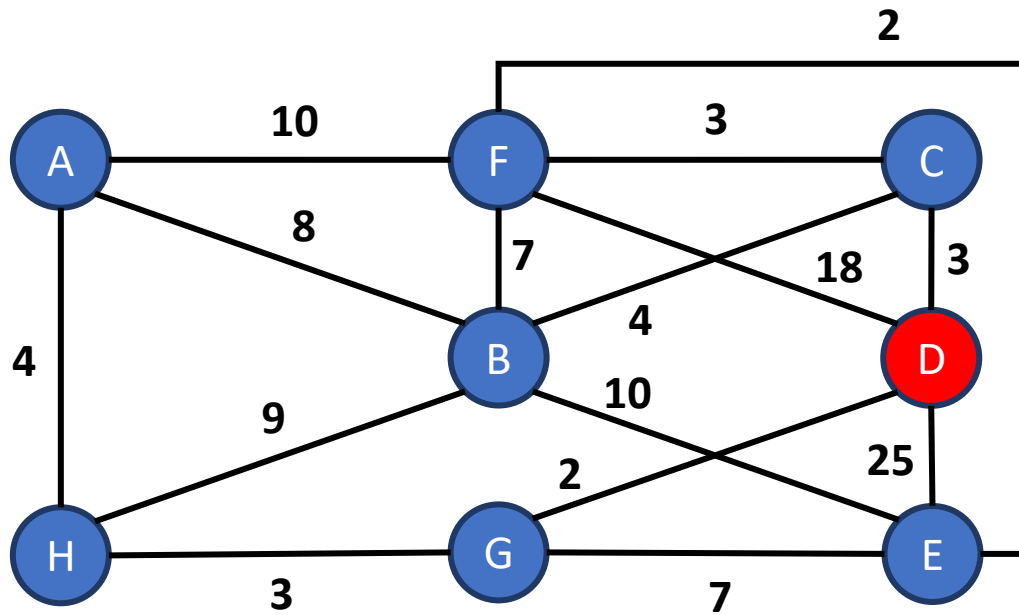
Prim's Algorithm

Existing Cable



v	Selected	d_v	π_v
A			
B			
C			
D			
E			
F			
G			
H			

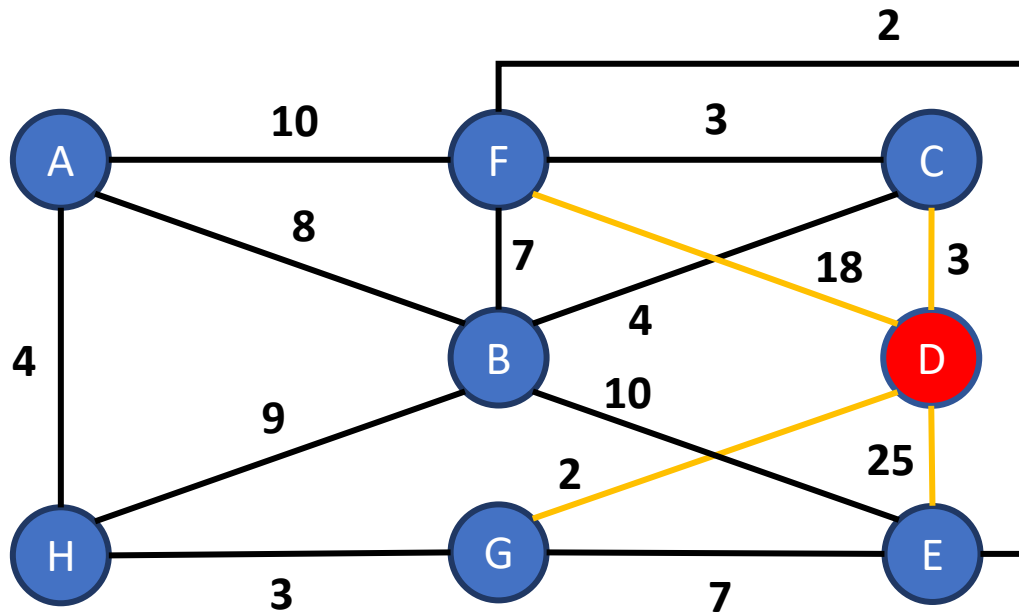
Prim's Algorithm



v	Selected	d_v	π_v
A			
B			
C			
D	T	0	-
E			
F			
G			
H			

Arbitrarily select a vertex and mark as selected.

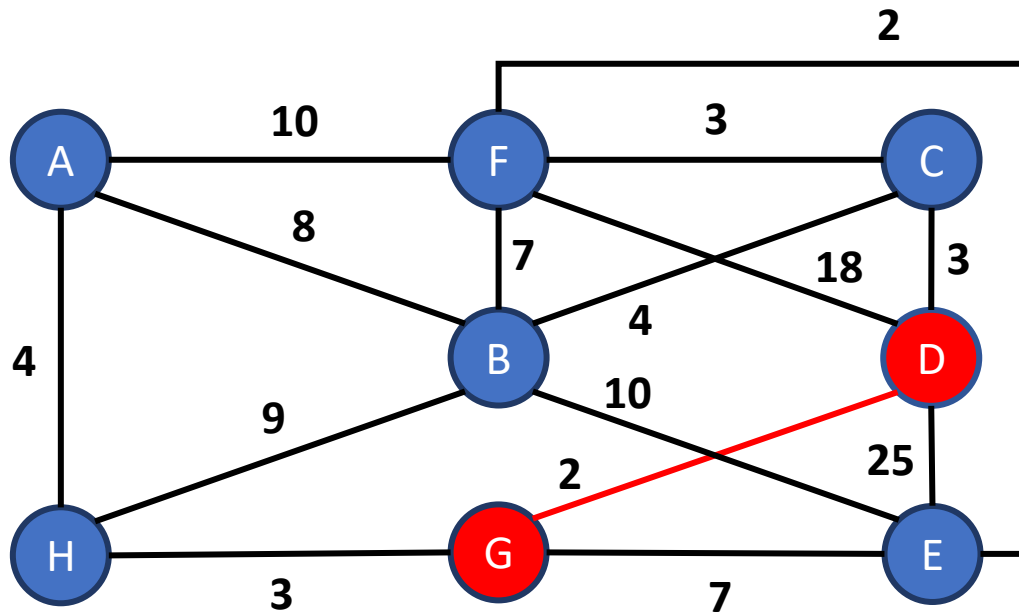
Prim's Algorithm



Update distances of adjacent, unselected vertices

v	Selected	d_v	π_v
A			
B			
C		3	D
D	T	0	-
E		25	D
F		18	D
G		2	D
H			

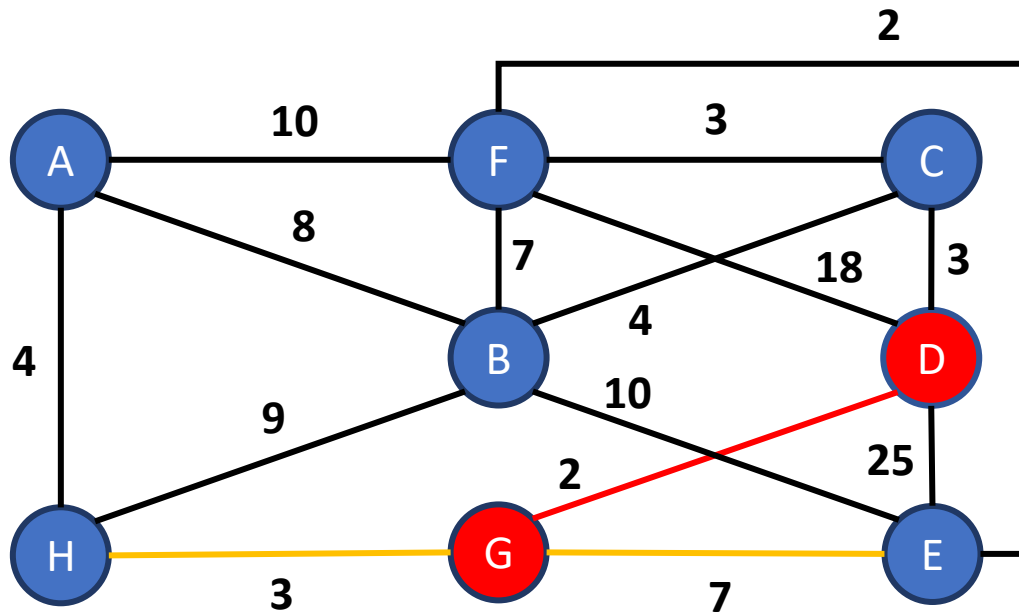
Prim's Algorithm



Select reachable vertex with minimum distance

v	Selected	d_v	π_v
A			
B			
C		3	D
D	T	0	-
E		25 7	D G
F		18	D
G	T	2	D
H		3	G

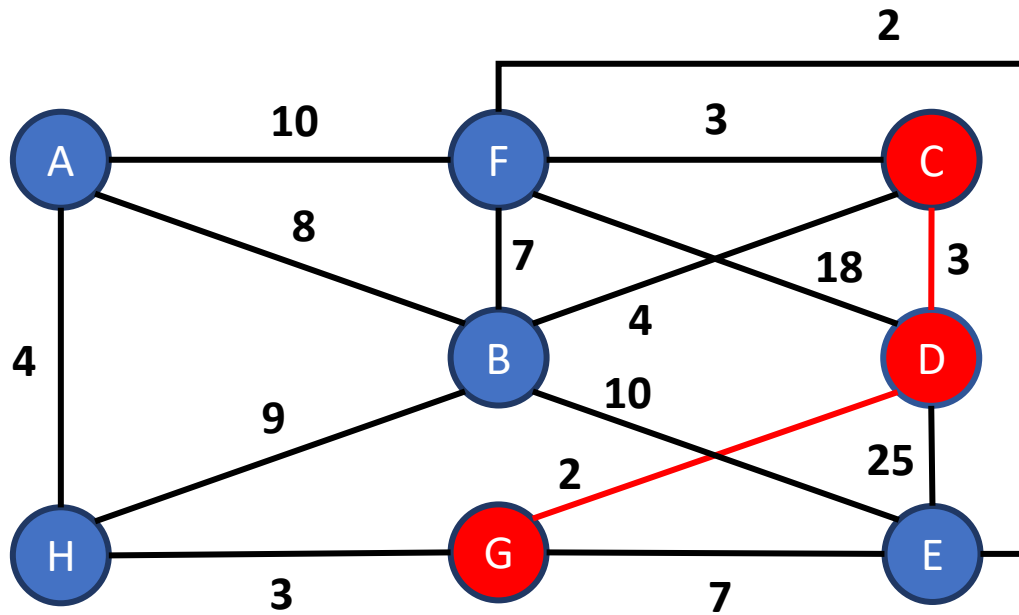
Prim's Algorithm



Update distances of adjacent, unselected vertices

v	Selected	d_v	π_v
A			
B			
✓ C		3	D
D	T	0	-
✓ E		25 7	D G
✓ F		18	D
G	T	2	D
✓ H		3	G

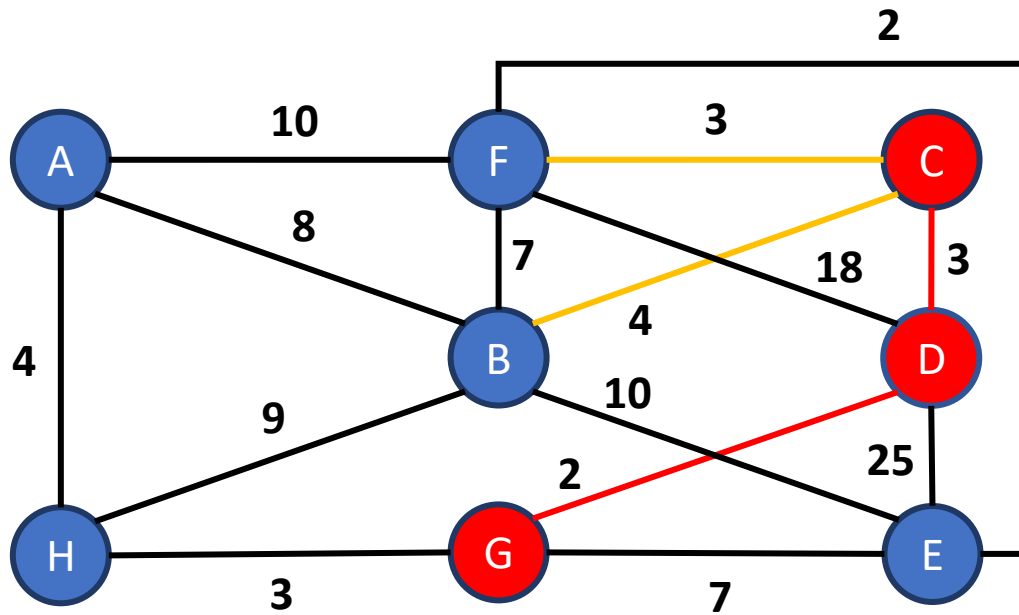
Prim's Algorithm



Select reachable vertex with minimum distance

v	Selected	d_v	π_v
A			
B		4	C
C	T	3	D
D	T	0	-
E		25 7	D G
F		18 3	D C
G	T	2	D
H		3	G

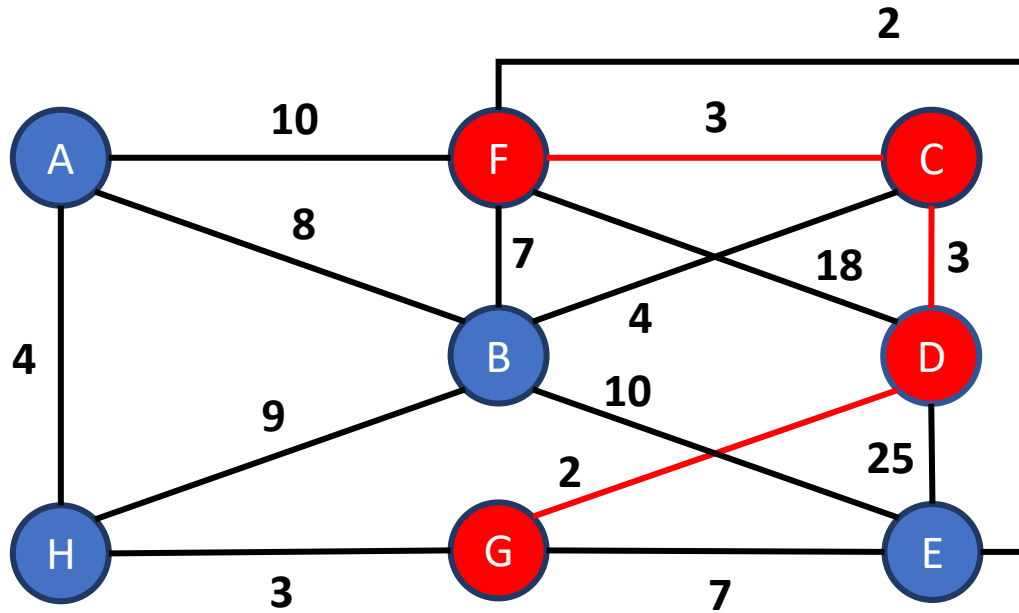
Prim's Algorithm



Update distances of adjacent, unselected vertices

v	Selected	d_v	π_v
A			
✓ B		4	C
C	T	3	D
D	T	0	-
✓ E		25 7	G
✓ F		18 3	D C
G	T	2	D
✓ H		3	G

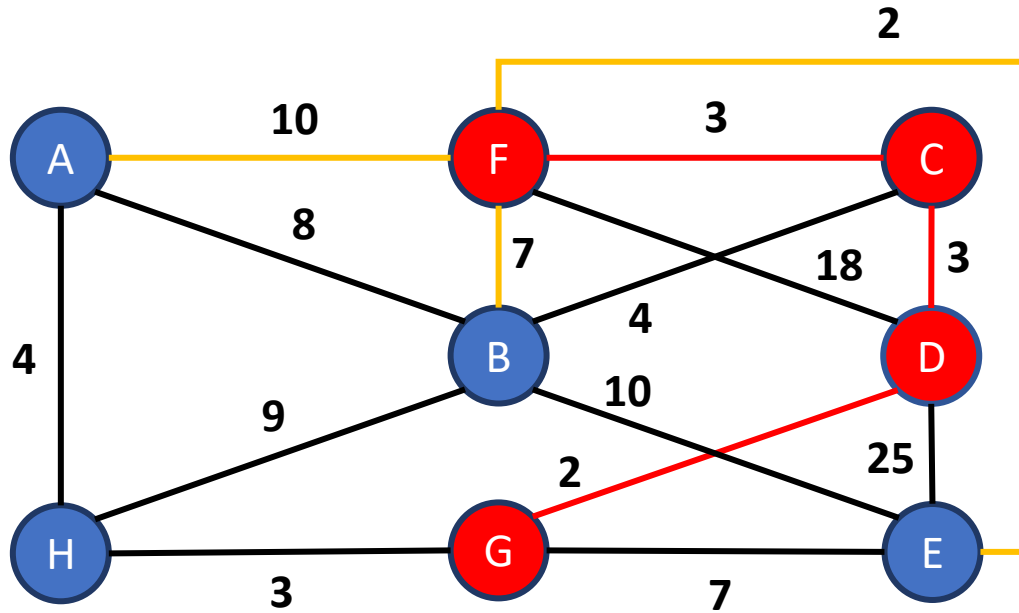
Prim's Algorithm



Select reachable vertex with minimum distance

v	Selected	d_v	π_v
A			
B		4	C
C	T	3	D
D	T	0	-
E		25 7	G
F	T	18 3	D C
G	T	2	D
H		3	G

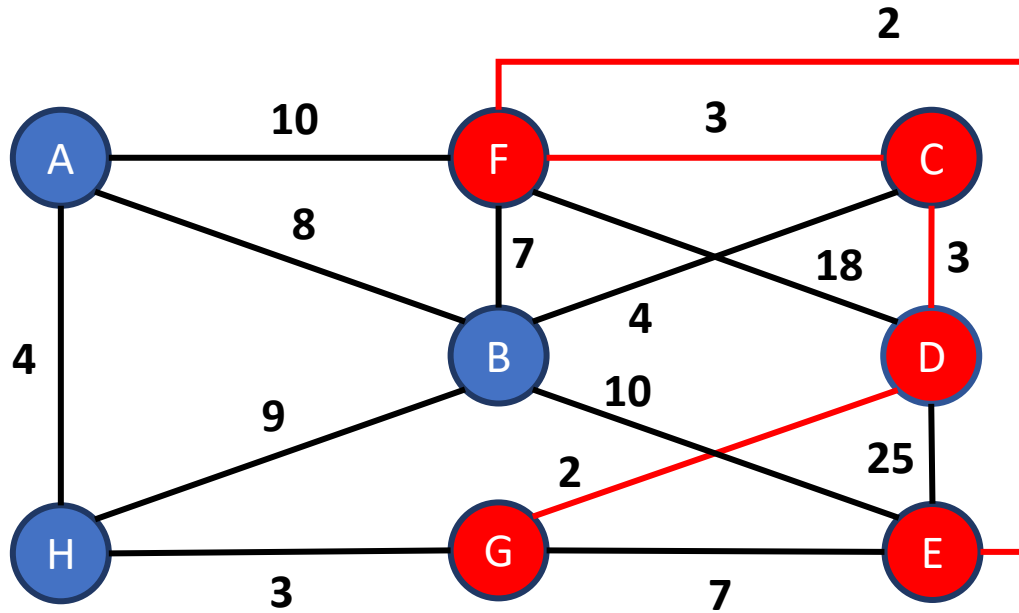
Prim's Algorithm



Update distances of adjacent, unselected vertices

v	Selected	d_v	π_v
✓ A		10	F
✓ B		4	C
C	T	3	D
D	T	0	-
✓ E		25 7 2	G F
F	T	18 3	D C
G	T	2	D
✓ H		3	G

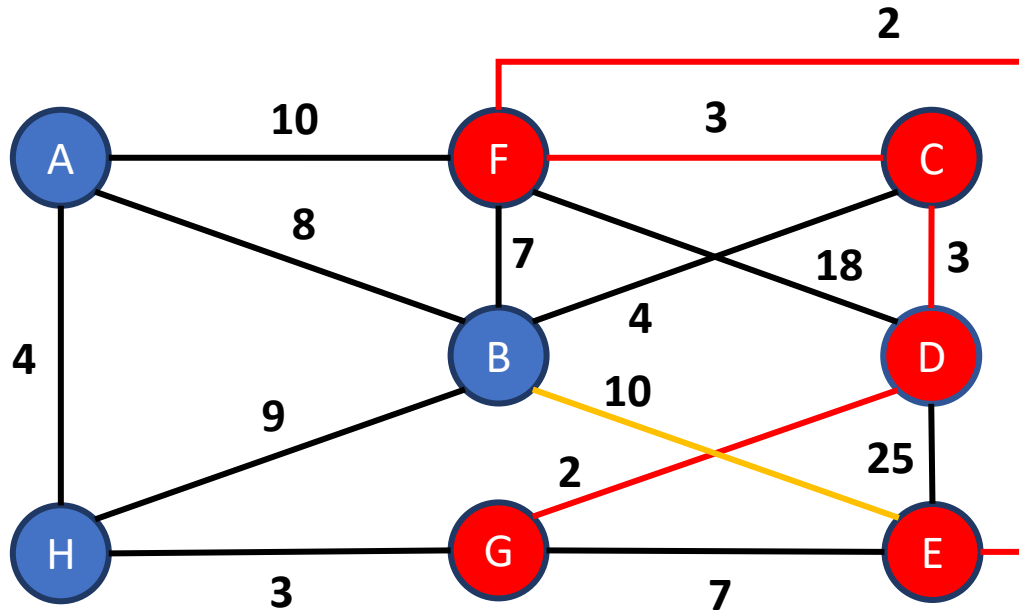
Prim's Algorithm



Select reachable vertex with minimum distance

v	Selected	d_v	π_v
A		10	F
B		4	C
C	T	3	D
D	T	0	-
E	T	25 7 2	G F
F	T	18 3	D C
G	T	2	D
H		3	G

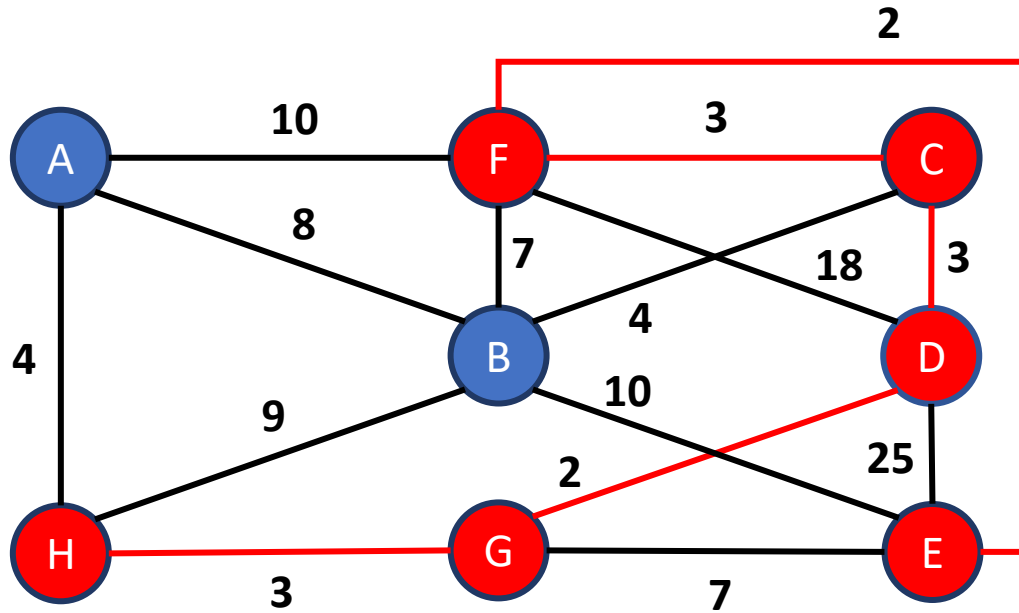
Prim's Algorithm



Update distances of adjacent, unselected vertices

v	Selected	d_v	π_v
✓ A		10	F
✓ B		4	C
C	T	3	D
D	T	0	-
E	T	25 7 2	G F
F	T	18 3	D C
G	T	2	D
✓ H		3	G

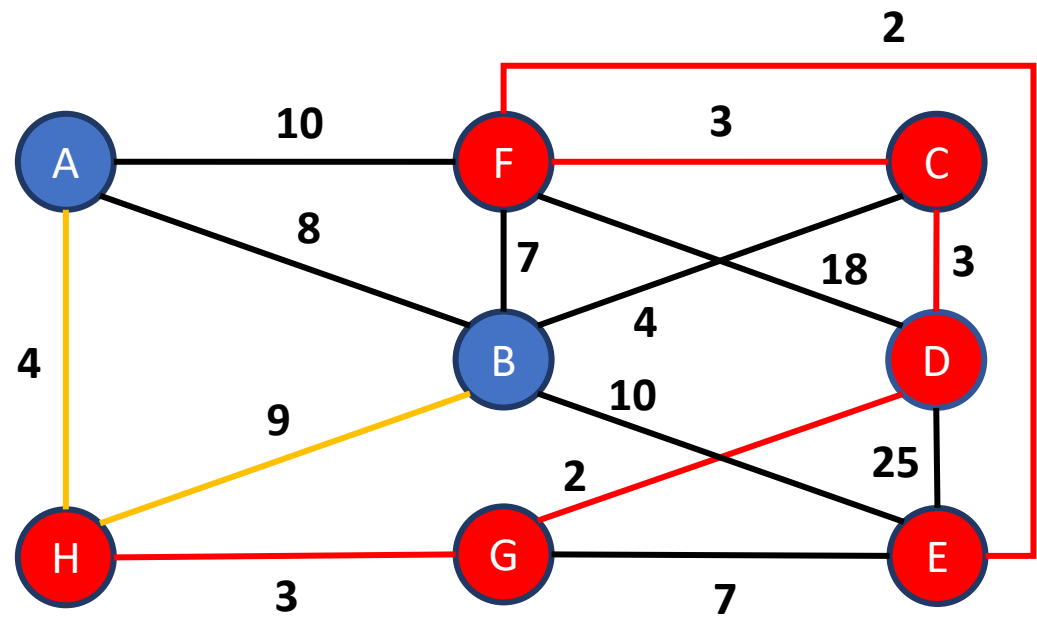
Prim's Algorithm



Select reachable vertex with minimum distance

v	Selected	d_v	π_v
A		10 4	F H
B		4	C
C	T	3	D
D	T	0	-
E	T	25 7 2	G F
F	T	18 3	D C
G	T	2	D
H	T	3	G

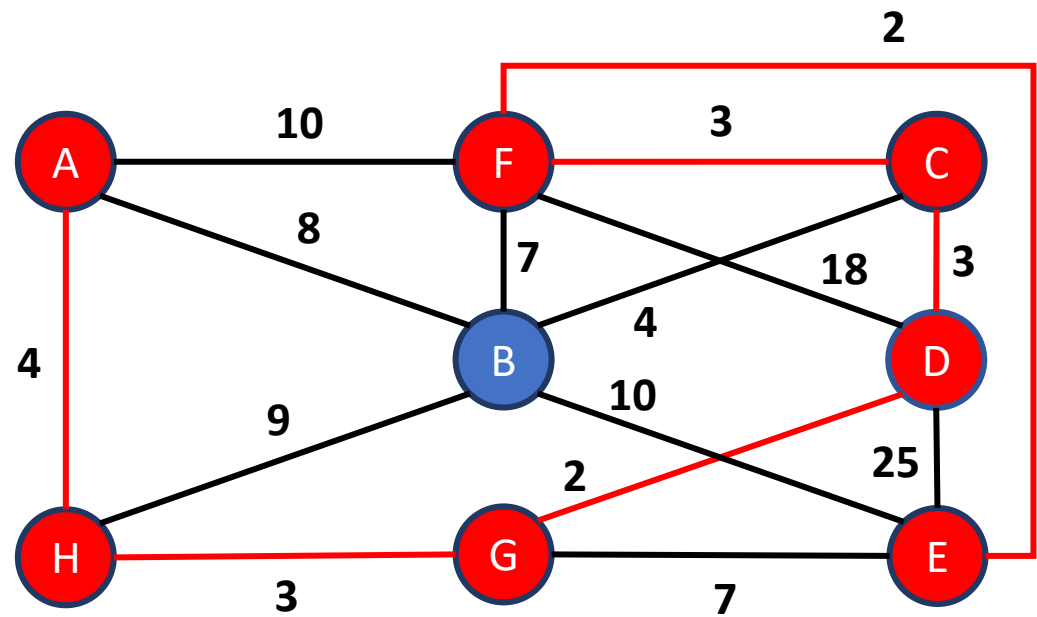
Prim's Algorithm



v	Selected	d_v	π_v
✓ A		10 4	F H
✓ B		4	C
C	T	3	D
D	T	0	-
E	T	25 7 2	G F
F	T	18 3	D C
G	T	2	D
H	T	3	G

Update distances of adjacent, unselected vertices

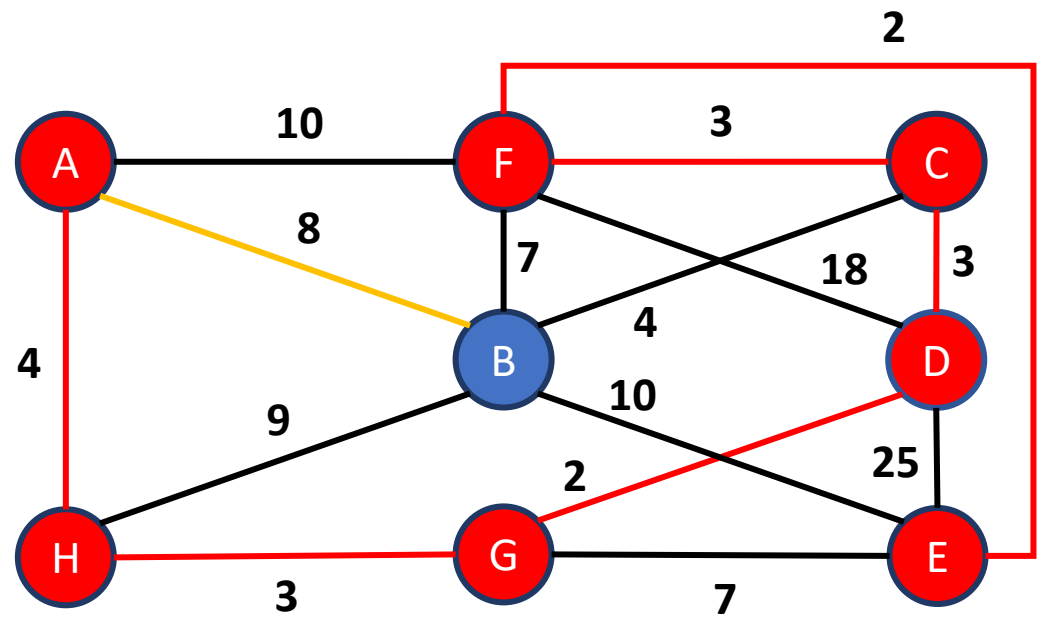
Prim's Algorithm



v	Selected	d_v	π_v
A	T	10 4	F H
B		4	C
C	T	3	D
D	T	0	-
E	T	25 7 2	G F
F	T	18 3	D C
G	T	2	D
H	T	3	G

Select reachable vertex with minimum distance

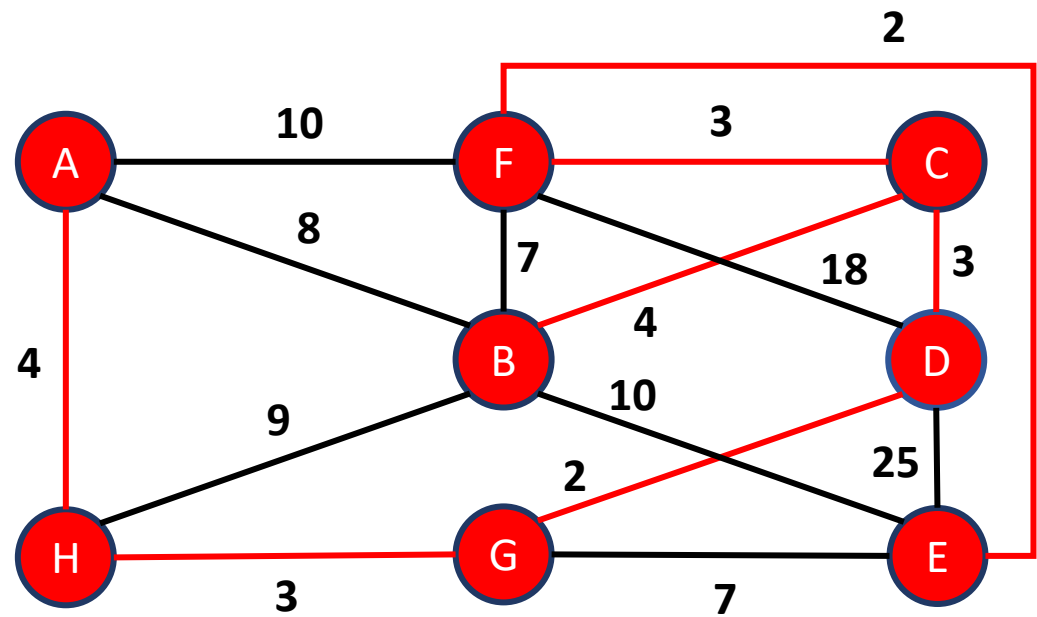
Prim's Algorithm



v	Selected	d_v	π_v
A	T	10 4	F H
B		4	C
C	T	3	D
D	T	0	-
E	T	25 7 2	G F
F	T	18 3	D C
G	T	2	D
H	T	3	G

Update distances of adjacent, unselected vertices

Prim's Algorithm



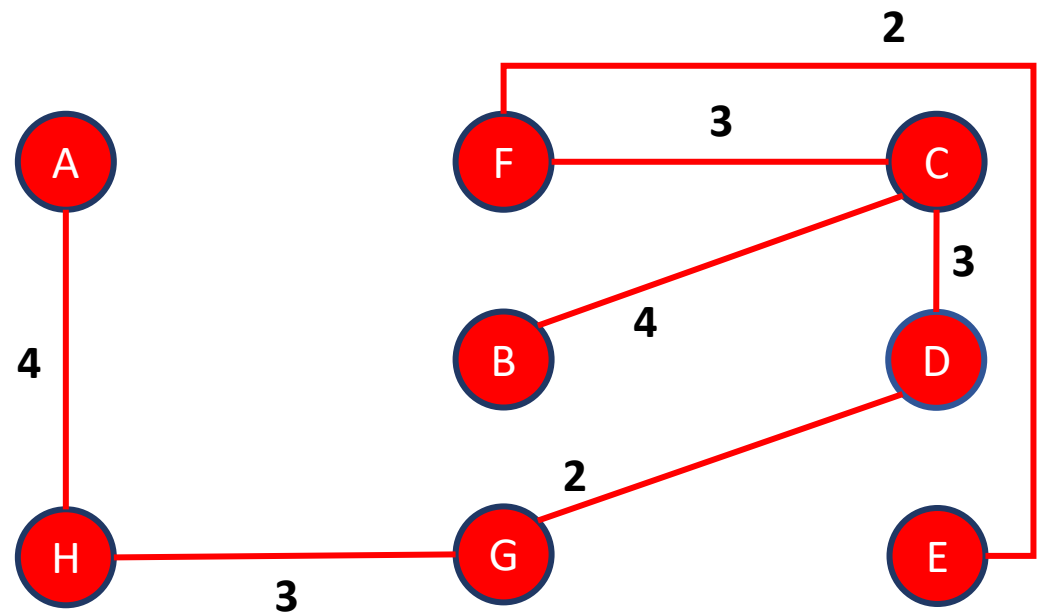
v	Selected	d_v	π_v
A	T	10 4	F H
B	T	4	C
C	T	3	D
D	T	0	-
E	T	25 7 2	G F
F	T	18 3	D C
G	T	2	D
H	T	3	G

Select reachable vertex with minimum distance

Prim's Algorithm

complete

$$\sum_v d_v = \underline{\underline{21}}$$



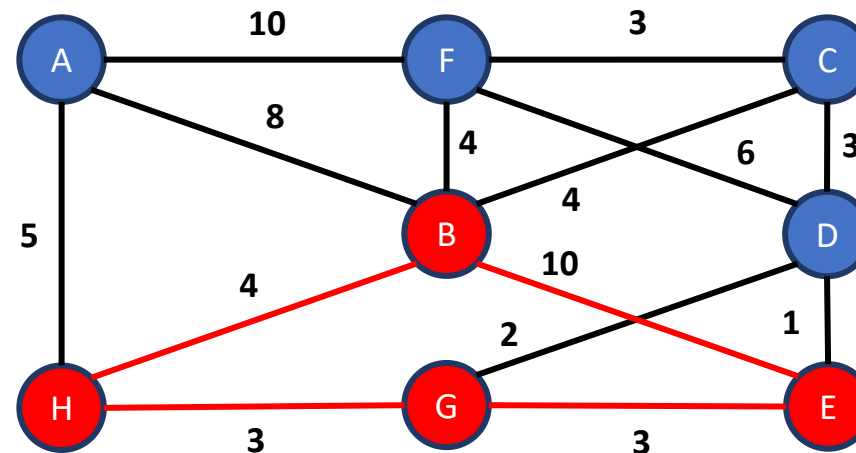
v	Selected	d_v	π_v
A	T	10 4	F H
B	T	4	C
C	T	3	D
D	T	0	-
E	T	25 7 2	G F
F	T	18 3	D C
G	T	2	D
H	T	3	G

All vertices added and we have the minimum spanning tree $\sum_v d_v = 21$

Kruskal's Algorithm

ORDER TIME

1. Order the edges by weightings, lowest to highest
2. Select the lowest vertex not selected if it does not form a cycle label as selected
3. Repeat step 2 until all vertices have been selected

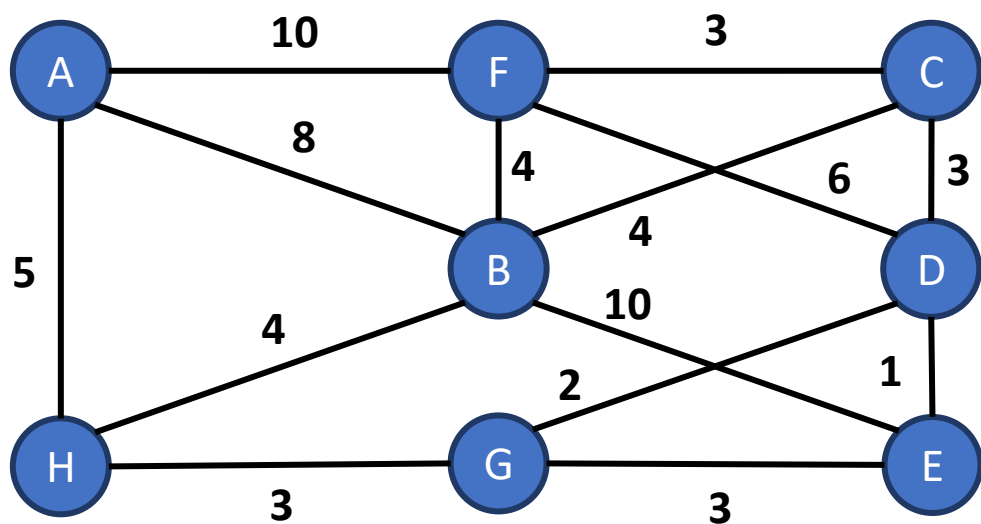


This is a cycle

Kruskal's Algorithm

$V = \{A, B, \dots, H\}$

Cardinality
of set

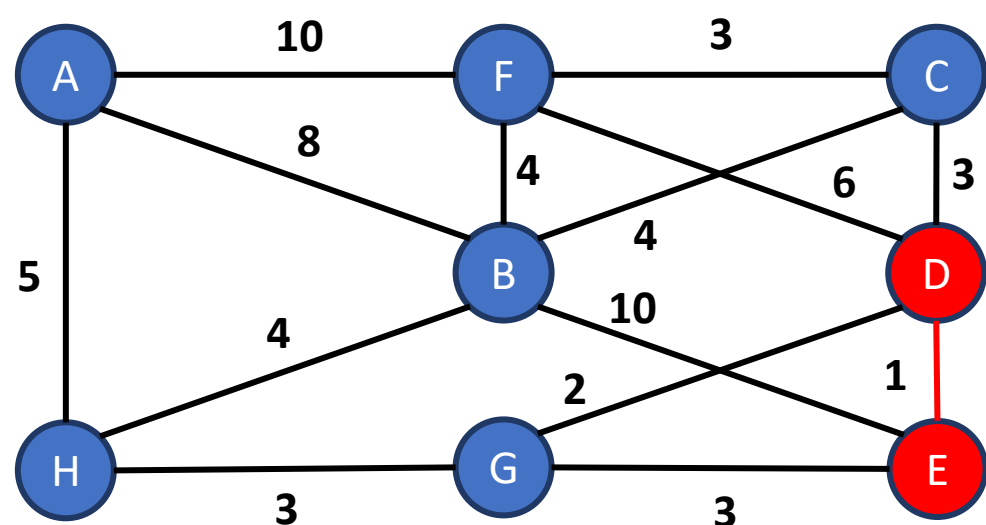


$$|V| = 8$$

Select first $|V| - 1$ edges that do not form a cycle

Edge e	w_e	Selected
(D,E)	1	
(D,G)	2	
(E,G)	3	
(C,D)	3	
(G,H)	3	
(C,F)	3	
(B,C)	4	
(B,E)	4	
(B,F)	4	
(B,H)	4	
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

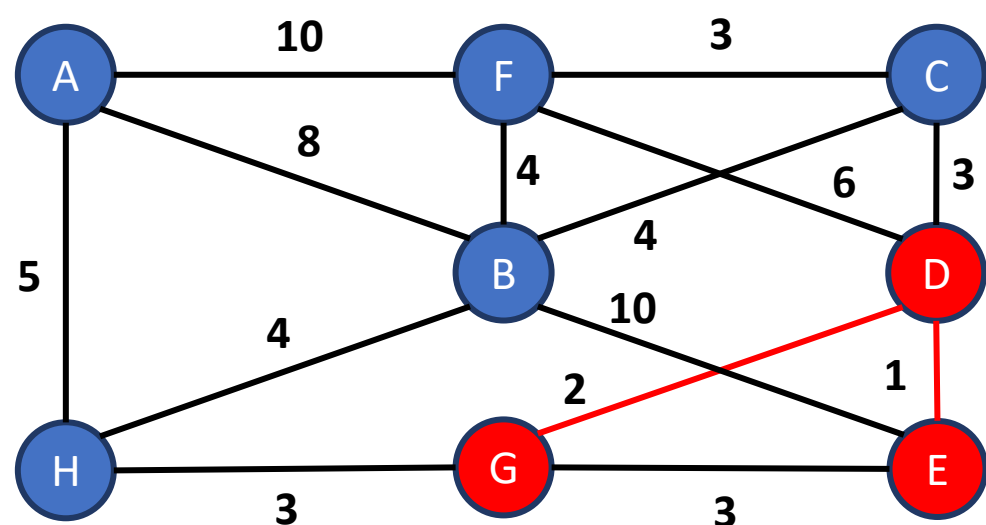
Kruskal's Algorithm



Select first $|V| - 1$ edges that do not form a cycle

Edge e	w_e	Selected
(D,E)	1	T
(D,G)	2	
(E,G)	3	
(C,D)	3	
(G,H)	3	
(C,F)	3	
(B,C)	4	
(B,E)	4	
(B,F)	4	
(B,H)	4	
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

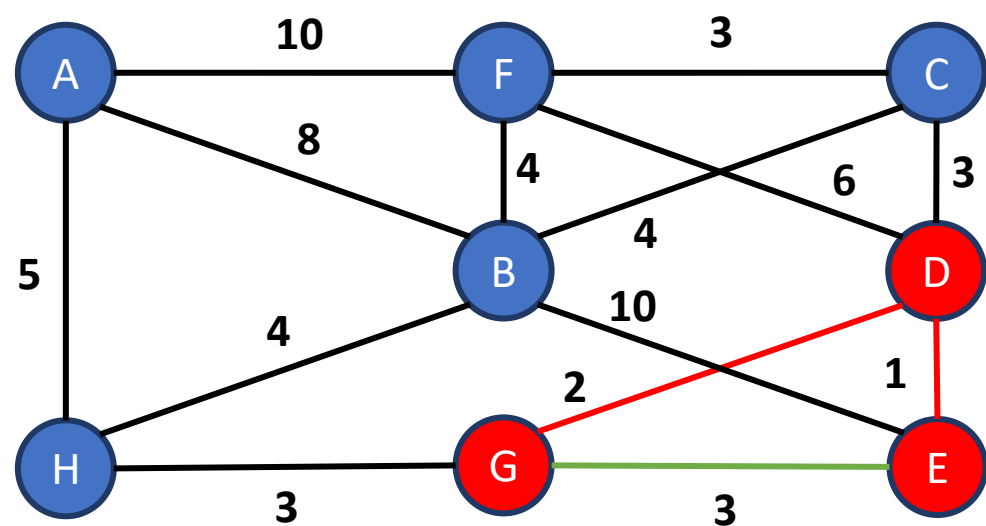
Kruskal's Algorithm



Select first $|V| - 1$ edges that do not form a cycle

Edge	d_v	Selected
(D,E)	1	T
(D,G)	2	T
(E,G)	3	
(C,D)	3	
(G,H)	3	
(C,F)	3	
(B,C)	4	
(B,E)	4	
(B,F)	4	
(B,H)	4	
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

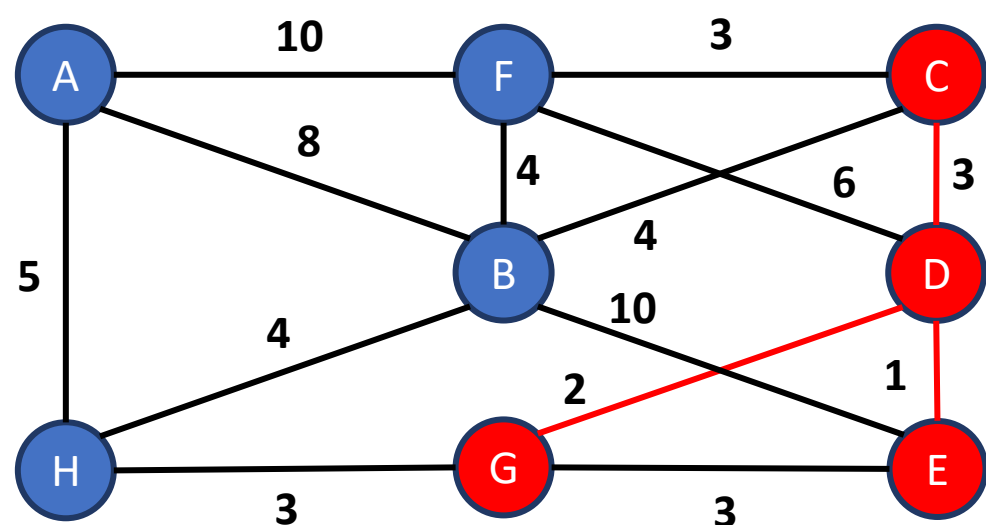
Kruskal's Algorithm



Select first $|V| - 1$ edges that do not form a cycle

Edge e	w_e	Selected
(D,E)	1	T
(D,G)	2	T
(E,G)	3	F
(C,D)	3	
(G,H)	3	
(C,F)	3	
(B,C)	4	
(B,E)	4	
(B,F)	4	
(B,H)	4	
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

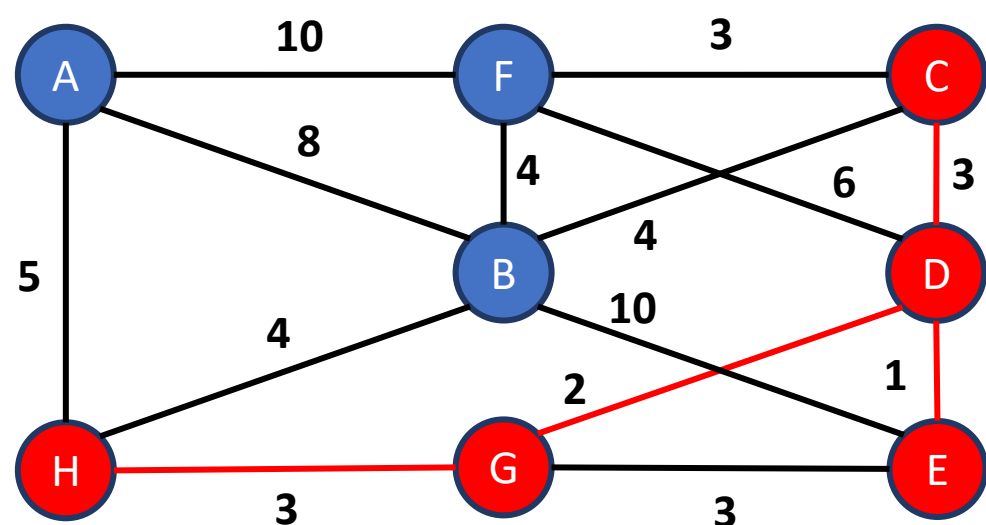
Kruskal's Algorithm



Select first $|V| - 1$ edges that do not form a cycle

Edge e	w_e	Selected
(D,E)	1	T
(D,G)	2	T
(E,G)	3	F
(C,D)	3	T
(G,H)	3	
(C,F)	3	
(B,C)	4	
(B,E)	4	
(B,F)	4	
(B,H)	4	
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

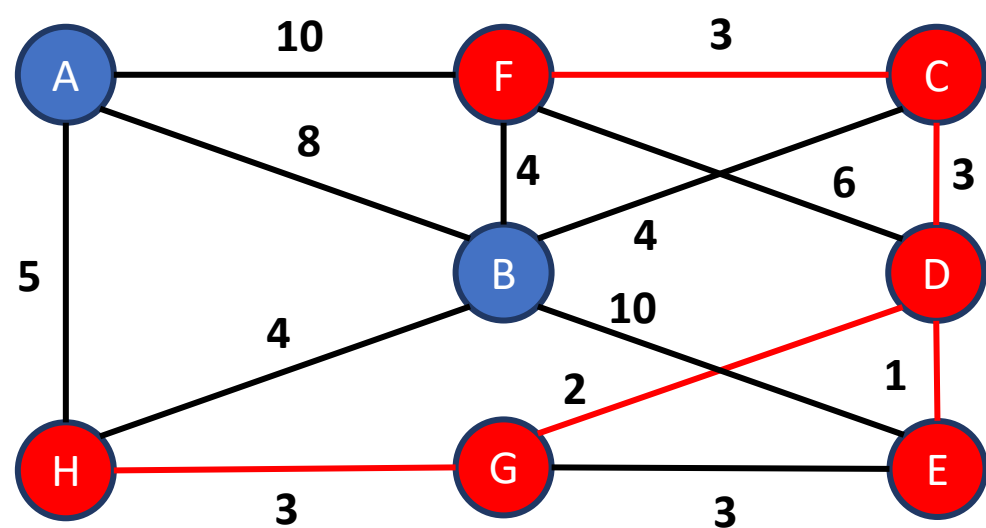
Kruskal's Algorithm



Select first $|V| - 1$ edges that do not form a cycle

Edge e	w_e	Selected
(D,E)	1	T
(D,G)	2	T
(E,G)	3	F
(C,D)	3	T
(G,H)	3	T
(C,F)	3	
(B,C)	4	
(B,E)	4	
(B,F)	4	
(B,H)	4	
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

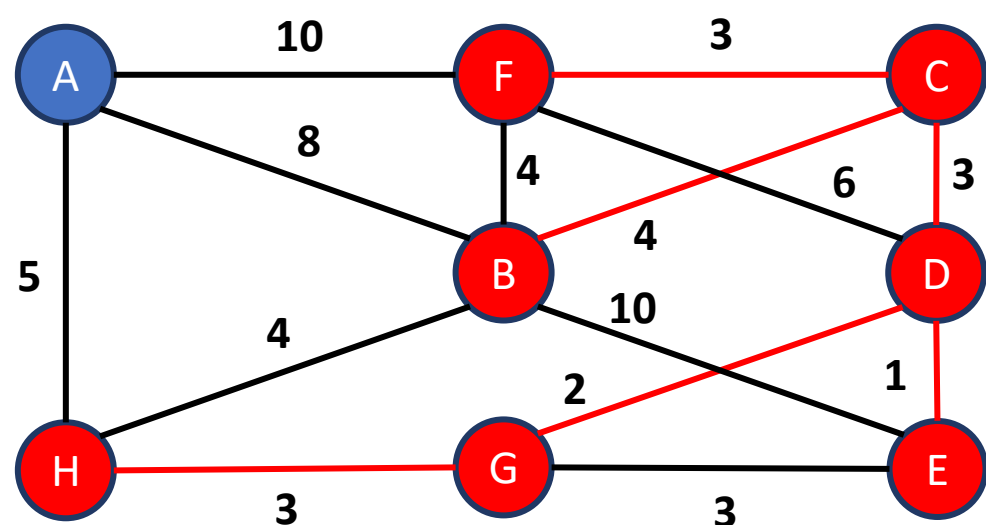
Kruskal's Algorithm



Select first $|V| - 1$ edges that do not form a cycle

Edge e	w_e	Selected
(D,E)	1	T
(D,G)	2	T
(E,G)	3	F
(C,D)	3	T
(G,H)	3	T
(C,F)	3	T
(B,C)	4	
(B,E)	4	
(B,F)	4	
(B,H)	4	
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

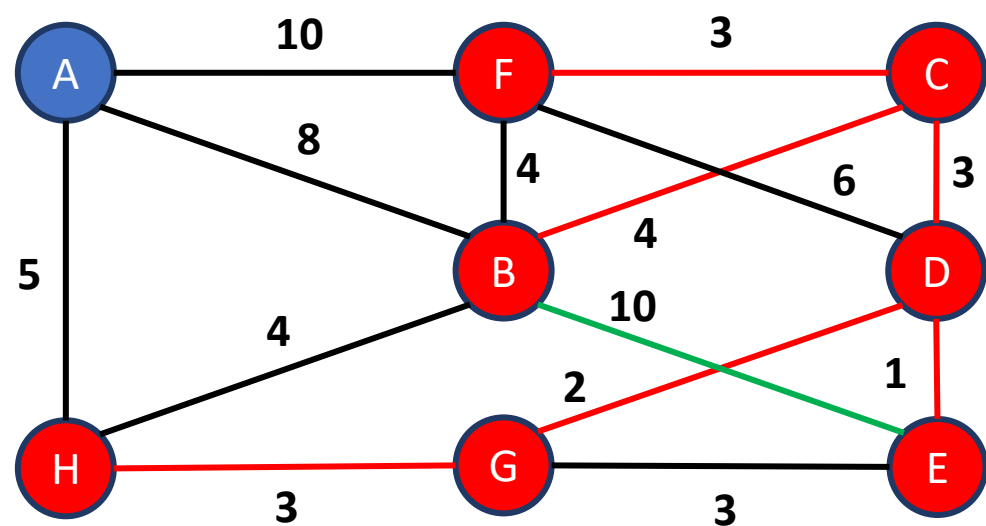
Kruskal's Algorithm



Select first $|V| - 1$ edges that do not form a cycle

Edge e	w_e	Selected
(D,E)	1	T
(D,G)	2	T
(E,G)	3	F
(C,D)	3	T
(G,H)	3	T
(C,F)	3	T
(B,C)	4	T
(B,E)	4	
(B,F)	4	
(B,H)	4	
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

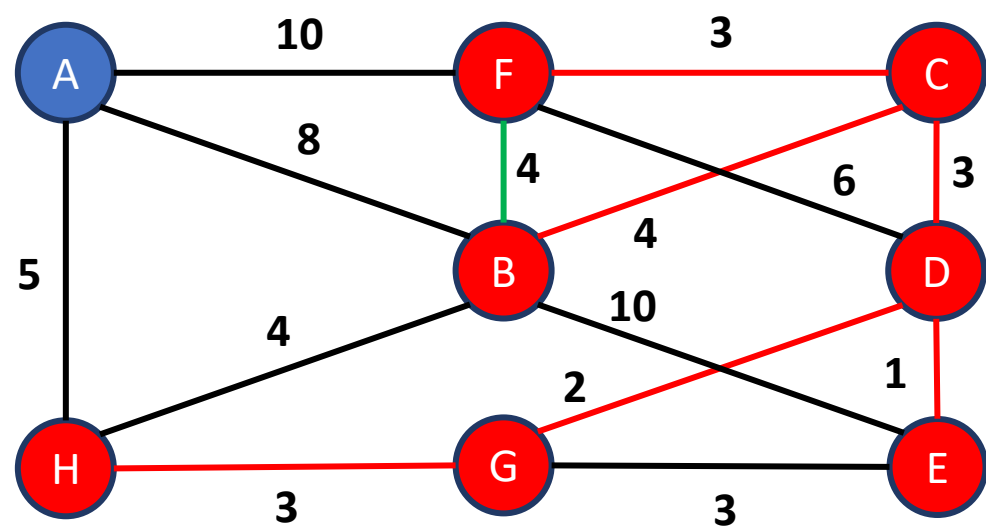
Kruskal's Algorithm



Select first $|V| - 1$ edges that do not form a cycle

Edge e	w_e	Selected
(D,E)	1	T
(D,G)	2	T
(E,G)	3	F
(C,D)	3	T
(G,H)	3	T
(C,F)	3	T
(B,C)	4	T
(B,E)	4	F
(B,F)	4	
(B,H)	4	
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

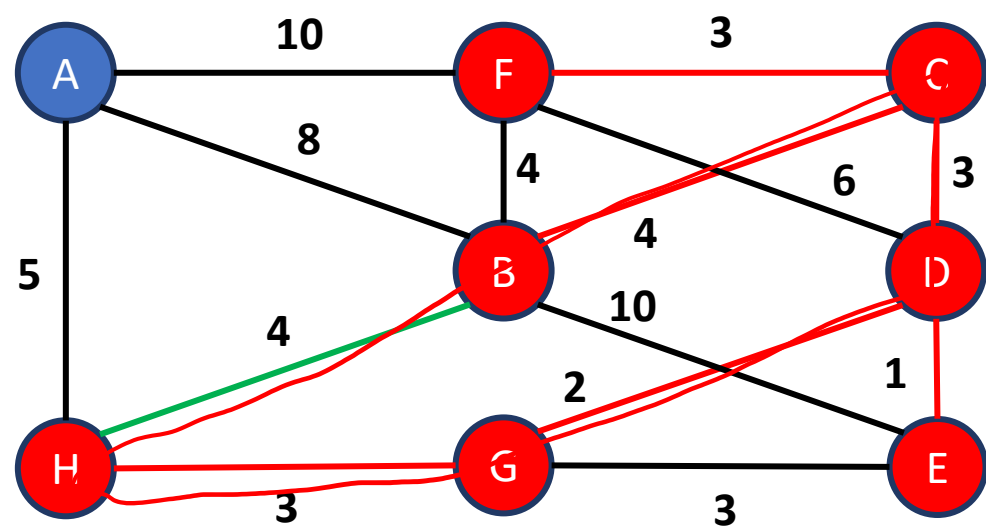
Kruskal's Algorithm



Select first $|V| - 1$ edges that do not form a cycle

Edge e	w_e	Selected
(D,E)	1	T
(D,G)	2	T
(E,G)	3	F
(C,D)	3	T
(G,H)	3	T
(C,F)	3	T
(B,C)	4	T
(B,E)	4	F
(B,F)	4	F
(B,H)	4	
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

Kruskal's Algorithm

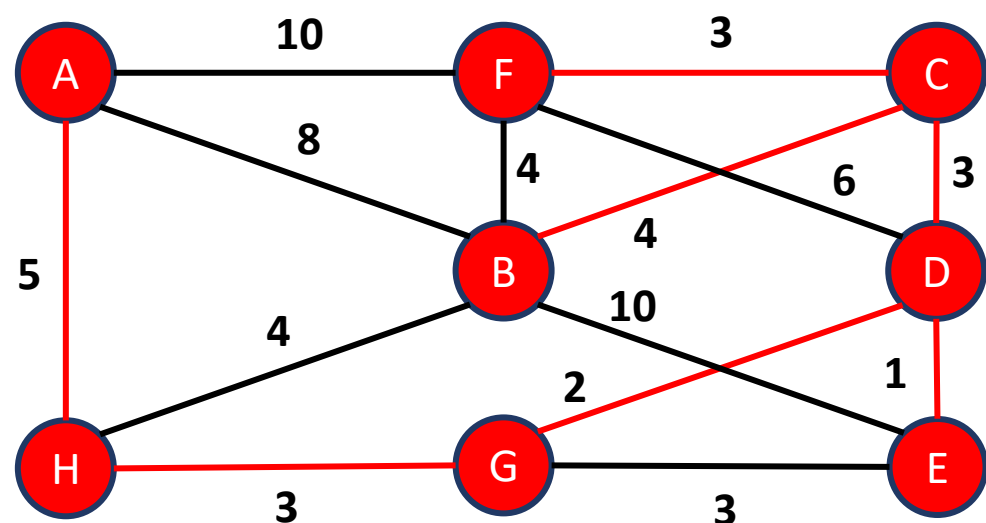


Forms a cycle!

Select first $|V| - 1$ edges that do not form a cycle

Edge e	w_e	Selected
(D,E)	1	T
(D,G)	2	T
(E,G)	3	F
(C,D)	3	T
(G,H)	3	T
(C,F)	3	T
(B,C)	4	T
(B,E)	4	F
(B,F)	4	F
(B,H)	4	F
(A,H)	5	
(D,F)	6	
(A,B)	8	
(A,F)	10	

Kruskal's Algorithm



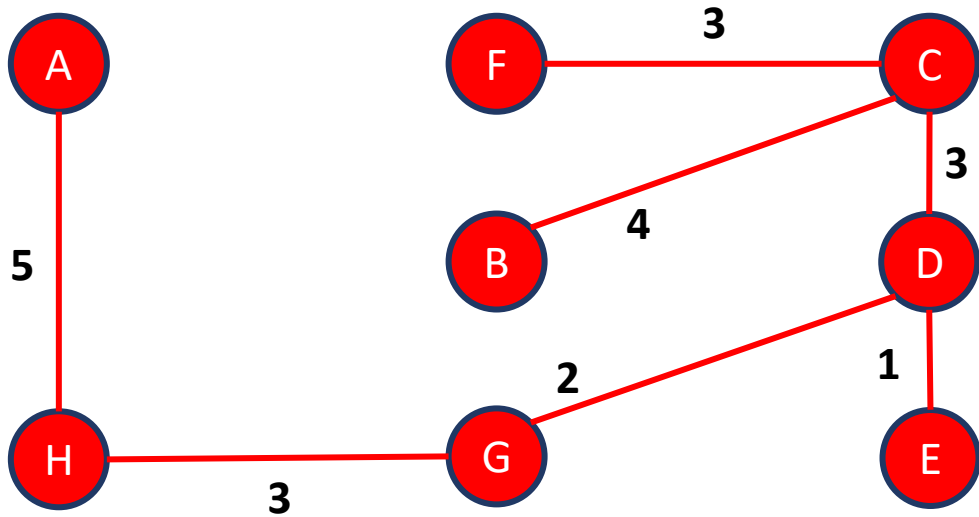
All vertices are selected. Stop

Select first $|V| - 1$ edges that do not form a cycle

7 edges
 $|V| - 1$ edges

Edge e	w_e	Selected
(D,E)	1	T
(D,G)	2	T
(E,G)	3	F
(C,D)	3	T
(G,H)	3	T
(C,F)	3	T
(B,C)	4	T
(B,E)	4	F
(B,F)	4	F
(B,H)	4	F
(A,H)	5	T
(D,F)	6	
(A,B)	8	
(A,F)	10	

Kruskal's Algorithm



$|V| - 1$ edges selected, let A be the edges in the MST, then

$$\sum_{e \in A} w_e = 21$$

$\sum_e w_e$

Edge e	w_e	Selected
(D,E)	1	T ✓
(D,G)	2	T ✓
(E,G)	3	F
(C,D)	3	T ✓
(G,H)	3	T ✓
(C,F)	3	T ✓
(B,C)	4	T ✓
(B,E)	4	F
(B,F)	4	F
(B,H)	4	F
(A,H)	5	T ✓
(D,F)	6	F
(A,B)	8	F
(A,F)	10	F

References

A red bracket is positioned to the left of the first reference, grouping the authors and the year together.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C.
(2001). *Introduction to Algorithms*. The MIT Press. ISBN: 0262032937

Taha, Hamdy. A - Operations Research: An Introduction (Tenth Edition)